

Contents

1 Extra	1
2 Data Structures	1
2.1 BIT (Fenwick Tree)	1
2.2 BIT (Fenwick Tree) 2D	1
2.3 DSU - Disjoint Set Union	1
2.4 DSU - Binary Tree	1
2.5 MinQueue	1
2.6 Mo's Algorithm	2
2.7 Mo with updates	2
2.8 Segment Tree	2
2.8.1 Iterative Point Update	2
2.8.2 Iterative Lazy	3
2.8.3 Recursive Lazy	3
2.9 Sparse Table RMQ	4
3 Graphs	4
3.1 BFS 0-1	4
3.2 Block Cut Tree (BCT)	4
3.3 Bridges and Articulation points	4
3.4 Dijkstra	5
3.5 Dinic - Flow/matchings	5
3.6 Floyd-Warshall	5
3.7 Hopcroft-Karp - Bipartite Matching	6
3.8 Hungarian	6
3.9 Kuhn - Bipartite Matching	6
3.10 Min cost flow	6
3.11 MST - Kruskal	7
3.12 MST - Prim	7
3.13 SCC compressing	7
4 DP	8
4.1 Bin Packing	8
4.2 Broken Profile DP	8
4.3 Convex Hull Trick (CHT)	8
4.4 Edit Distance (Levenshtein)	9
4.5 Knapsack - 1D	9
4.6 Knapsack - 2D	9
4.7 LCS - Longest Common Subsequence	9
4.8 LiChao Tree	9
4.9 LIS - Longest Increasing Subsequence	10
4.10 SOSDP	10
4.11 Subset Sum	10
4.12 Subset Sum with Bitset	10

5 Trees	1
5.1 Centroid Decomposition	1
5.2 Sum of distances	1
5.3 Edge HLD	1
5.4 HLD - Heavy light decomposition	1
5.5 LCA - RMQ	1
5.6 LCA - binary lifting	1
6 Problemas clássicos	1
6.1 2SAT	1
6.2 Next Greater Element	1
7 Strings	1
7.1 Hashing	1
7.2 KMP	1
7.3 Suffix Array	1
7.4 Suffix Automaton	1
7.5 Z	1
8 Math	1
8.1 Combinatorics (Pascal's Triangle)	1
8.1.1 Combinatorial Analysis	1
8.2 Convolutions	1
8.2.1 AND convolution	1
8.2.2 GCD convolution	1
8.2.3 LCM convolution	1
8.2.4 OR convolution	1
8.2.5 XOR convolution	1
8.3 Extended Euclid	1
8.4 Factorization	1
8.5 FFT - Fast Fourier Transform	1
8.6 Inclusion-Exclusion Principle	1
8.7 Legendre's formula	1
8.8 Linear Sieve + Möbius Function	1
8.8.1 Applications	1
8.9 Matrix template	1
8.10 Mint	1
8.11 Modular Inverse	1
8.12 Number Theoretic Transform (NTT)	1
8.12.1 NTT-Friendly Primes and Roots	1
8.13 Euler's Totient	1
9 Geometry	1
9.1 Convex hull - Graham Scan	1
9.2 Basic elements - geometry lib	1
9.2.1 Polygon Area	1
9.2.2 Point in polygon	1

10 1 Extra

11 2 Data Structures

11 2.1 BIT (Fenwick Tree)

Data structure that can handle point update and range queries for invertible operations. Obs: If the operation is not invertible (e.g. min,max,gcd), it can only answer prefix queries

```

1 // [l,r] interval convention and 0-indexed parameters
2 struct BIT {
3     vector<ll> bit;
4     int n;
5     BIT(int _n = 0){
6         n = _n;
7         bit.assign(n+1,0ll);
8     }
9     ll presum(int i){
10         ll ans = 0;
11         for (++i; i > 0; i -= i&-1)
12             ans += bit[i];
13         return ans;
14     }
15     ll sum(int l, int r){
16         return presum(r) - presum(l-1);
17     }
18     void add(int i, ll v){
19         for (++i; i <= n; i += i&-1)
20             bit[i] += v;
21     }
22 };

```

17 2.2 BIT (Fenwick Tree) 2D

2D Sum BIT, update and sum. The struct encapsulates the 1-based indexing, therefore parameters are 0-based indexed.

Query/update time: $\mathcal{O}(\log n \cdot \log m)$

Construction time (using proper constructor): $\mathcal{O}(nm)$

Space: $\mathcal{O}(nm)$

```

1 // [l,r] interval cnvention and 0-based indexing
2 struct BIT2D {
3     vector<vector<ll>> bit;
4     int n, m;
5     BIT2D(int _n, int _m){
6         n = _n; m = _m;
7         bit.assign(n+1,vector<ll>(m+1));
8     }
9     BIT2D(vector<vector<ll>> &a) {
10         n = a.size();
11         m = a[0].size();
12         bit.assign(n+1, vector<ll>(m+1));
13     }
14     for (int i = 1; i <= n; i++)
15         for (int j = 1; j <= m; j++)
16             bit[i][j] = a[i-1][j-1];

```

```

17
18     for (int i = 1; i <= n; i++) {
19         for (int j = 1; j <= m; j++) {
20             int nxt = j + (j&-j);
21             if (nxt <= m) bit[i][nxt] += bit[i][j];
22         }
23     }
24     for (int i = 1; i <= n; i++) {
25         int nxt = i + (i&-i);
26         if (nxt <= n) {
27             for (int j = 1; j <= m; j++)
28                 bit[nxt][j] += bit[i][j];
29         }
30     }
31 }
32 void add(int si, int sj, ll v){
33     for(int i = si+1; i <= n; i+=i&-i)
34         for (int j = sj+1; j <= m; j+=j&-j)
35             bit[i][j] += v;
36 }
37 ll presum(int si, int sj){
38     ll ans = 0;
39     for (int i = si+1; i > 0; i-=i&-i)
40         for (int j = sj+1; j > 0; j-=j&-j)
41             ans += bit[i][j];
42     return ans;
43 }
44 ll sum(ii ab, ii cd){
45     auto [a,b] = ab;
46     auto [c,d] = cd;
47     return presum(c,d) - presum(a-1,d) - presum(c,b-1)
48         + presum(a-1,b-1);
49 }

```

2.3 DSU - Disjoint Set Union

Query/update time: $\mathcal{O}(1)$

Construction time: $\mathcal{O}(n)$

Space: $\mathcal{O}(n)$

```

1 struct DSU {
2     vi p, sz;
3     DSU(int n) {
4         p.resize(n);
5         iota(p.begin(), p.end(), 0);
6         sz.assign(n, 1);
7     }
8     int find(int i) {
9         if (p[i] == i) return i;
10        return p[i] = find(p[i]);
11    }
12    bool join(int u, int v) {
13        u = find(u);
14        v = find(v);
15        if (u == v) return false;
16        if (sz[u] < sz[v]) swap(u, v);
17        p[v] = u;
18        sz[u] += sz[v];
19        return true;
20    }
21 };

```

2.4 DSU - Binary Tree

Specific code to find maximum path sums between pairs of vertices. Uses Kruskal-style MST. Query/update time: possibly $\mathcal{O}(n)$ Construction time: $\mathcal{O}(n)$ Space: $\mathcal{O}(n)$

```

1 vi d;
2 vi _ii e;
3 vi ans;
4
5 int merged;
6 vi _p, _leaf, _wei;
7 vvi adj;
8 int _find(int u) { return _p[u] == u ? u : _p[u] = _find
    (_p[u]); }
9 void _union(int u, int v, int w){
10     u = _find(u);
11     v = _find(v);
12     int merge_ind = merged+n;
13     _p[u] = merge_ind;
14     _p[v] = merge_ind;
15     _leaf[merge_ind] = _leaf[u] + _leaf[v];
16     _wei[merge_ind] = max(_wei[u], _wei[v]);
17     adj[u].push_back(merge_ind);
18     adj[merge_ind].push_back(u);
19     adj[v].push_back(merge_ind);
20     adj[merge_ind].push_back(v);
21     merged++;
22 }
23 void make(){
24     _p = vi(2*n);
25     for(int i = 0; i < 2*n; i++) _p[i] = i;
26     _leaf = vi(2*n, 1);
27     _wei = vi(2*n);
28     for(int i = 0; i < n; i++) _wei[i] = d[i];
29     merged = 0;
30     adj = vvi(2*n);
31 }
32
33 void dfs(int u, int p){
34     for(auto &v: adj[u]){
35         if(v == p) continue;
36         ans[v] = ans[u] + (_leaf[u] - _leaf[v])*_wei[u];
37         dfs(v, u);
38     }
39 }

```

2.5 MinQueue

Minimum (or maximum) queue. All operations are $\mathcal{O}(1)$ on average. Useful for sliding window max/min queries

```

1 template<typename T>
2 struct MinQueue{
3     deque<pair<T,int>> q;
4     int added = 0;
5     int removed = 0;
6     // returns [value, index]
7     pair<T,int> getmin(){ return q.front(); }
8
9     void push(T x){
10        // < for MaxQueue
11        while (!q.empty() && q.back().first > x) q.
            pop_back();
12        q.push_back({x, added++});
13    }

```

```

14 void pop(){
15     if (!q.empty() && q.front().second == removed) q.
        pop_front();
16     removed++;
17 }
18 bool empty() { return added == removed; }
19 };

```

2.6 Mo's Algorithm

A technique for solving offline range queries on static arrays by sorting queries to minimize total pointer movement. It processes intervals by incrementally updating the range via `add` and `remove` operations. With the optimal block size, the time complexity is $\mathcal{O}((N+Q)\sqrt{N})$ or $\mathcal{O}(N\sqrt{Q})$, depending on block size choice ($\sqrt{N}$ or $N/\sqrt{Q}$). This example solves queries for distinct elements in range

```

1 struct Mo {
2     struct Query {
3         int l, r, idx, b;
4         bool operator<(const Query& o) const {
5             return b != o.b ? b < o.b :
6                 (b & 1 ? r > o.r : r < o.r);
7         }
8     };
9
10    int n, block_sz;
11
12    // custom stuff
13    vi freq, a;
14    int ans = 0;
15
16    vector<Query> queries;
17    Mo(int n) : n(n), block_sz(round(sqrt(n))) {}
18
19    // [l,r] indexed
20    void add_query(int l, int r, int i) {
21        queries.push_back({l,r,i,1/block_sz});
22    }
23    void add(int i) {
24        // add val at i
25        freq[a[i]]++;
26        if (freq[a[i]] == 1) ans++;
27    }
28    void remove(int i) {
29        // remove value at i
30        freq[a[i]]--;
31        if (freq[a[i]] == 0) ans--;
32    }
33    int get_ans() {
34        // compute current answer
35        return ans;
36    }
37
38    vi run() {
39        vi ans(queries.size());
40        sort(queries.begin(), queries.end());
41        int l = 0, r = -1;
42        for (auto& q : queries) {
43            while (l > q.l) add(--l);
44            while (r < q.r) add(++r);
45            while (l < q.l) remove(l++);
46            while (r > q.r) remove(r--);
47            ans[q.idx] = get_ans();

```

```

48     }
49     return ans;
50 }
51 };

```

2.7 Mo with updates

Also sorts queries to minimize total pointer movement, but now there is an added dimension for time (POINT updates). $\mathcal{O}(QN^{2/3})$

```

1 struct MoUpdate {
2     int n;
3     const int block_sz;
4     struct Query {
5         int l,r,t,id;
6     };
7     vector<Query> queries;
8     vector<int> updates;
9     vector<int> a;
10
11     MoUpdate(vector<int> &v, int q) : block_sz(round(cbrt(
12         (2*v.size()*v.size())))) {
13         n = v.size() + q;
14         a = v;
15         init();
16     }
17     void add_query(int l, int r) {
18         queries.push_back({l,r,(int)updates.size(),(int)
19             queries.size()});
20     }
21     void add_update(int i, int x) {
22         updates.push_back({i,x});
23     }
24     void update(int l, int r, int t) {
25         auto &[i,v] = updates[t];
26         if (l <= i && i <= r) remove(i);
27         swap(a[i],v);
28         if (l <= i && i <= r) add(i);
29     }
30     vector<int> run() {
31         vector<int> ans(queries.size());
32         sort(queries.begin(),queries.end(), [&](Query qa,
33             Query qb){
34             int l = qa.l/block_sz;
35             int r = qa.r/block_sz;
36             int ql = qb.l/block_sz;
37             int qr = qb.r/block_sz;
38             using iii = tuple<int,int,int>;
39             return iii(l, l&1 ? -r:r, r&1 ? qa.t:-qa.t) <
40                 iii(ql, ql&1 ? -qr:qr, qr&1 ? qb.t:-qb.t);
41         });
42         int l = 0, r = -1, t = 0;
43         for (auto &q : queries) {
44             while(t < q.t) update(l,r,t++);
45             while(t > q.t) update(l,r,t--);
46             while(l > q.l) add(--l);
47             while(r < q.r) add(++r);
48             while(l < q.l) remove(l--);
49             while(r > q.r) remove(r--);
50             ans[q.id] = get_ans();
51         }
52         return ans;
53     }
54 }
55 // customize depending on problem
56 int ans = 0;

```

```

54 void init(){
55     // initialize custom stuff
56 }
57 void remove(int i) {
58     // remove a[i] from the current ans
59 }
60 void add(int i) {
61     // add a[i] to the current ans
62 }
63 int get_ans() {}
64 };

```

2.8 Segment Tree

You should (almost) never alter the `segtree` logic. Ideally you only modify the `node` struct by answering four questions:

1. `tag`: What data does a range update carry? (e.g., a single integer for addition, or a pair of integers a, d for an AP).
2. `apply()`: How does a `tag` mathematically alter the state of a single node?
3. `push()`: How does a parent node transfer its pending `tag` to its left and right children?
4. `combine()`: How do you calculate the state of a parent by merging the states of its children?

OBS: lazy segtrees must be called with `has=true`

2.8.1 Iterative Point Update

Best for: Standard point updates and range queries (e.g., Point Add + Range Sum).

- **Performance:** Best possible.
- **Memory:** Best possible: $2n$ allocation.
- **Use Case:** If no range updates, then use this.

```

1 // ITERATIVE - implemented for sum + point assignment
2 struct segtree {
3     struct node {
4         int sum = 0; // data
5         static node combine(const node& a, const node& b) {
6             return {a.sum + b.sum};
7         }
8         void apply(int v) {
9             sum = v;
10        }
11    };
12    int n;
13    vector<node> t;
14    void init(const vi& a) {
15        n = a.size();
16        t.assign(2*n, node());
17        for (int i = 0; i < n; i++) t[n + i].apply(a[i]);
18        for (int i = n-1; i > 0; i--) t[i] = node::combine(
19            t[i<<1], t[i<<1|1]);

```

```

19     }
20 void set(int p, int v) {
21     for (t[p += n].apply(v), p >= 1; p > 0; p >= 1) {
22         t[p] = node::combine(t[p<<1], t[p<<1|1]);
23     }
24 }
25 node query(int l, int r) {
26     node resL, resR;
27     for (l+=n, r+=n; l < r; l>=1, r>=1) {
28         if (l&1) resL = node::combine(resL, t[l++]);
29         if (r&1) resR = node::combine(t[--r], resR);
30     }
31     return node::combine(resL, resR);
32 }
33 };

```

2.8.2 Iterative Lazy

Best for: Index-independent range updates (e.g., Range Add, Range Set, Range Max, Range Flip).

- **Performance:** Noticeably faster than recursive lazy trees.
- **Index Blindness:** If the update depends on the range indexes themselves (such as AP addition), use the recursive lazy one.

```

1 // LAZY ITERATIVE - implemented for Hash + range
2 // assignment
3 // for reference, see DoubleHashing implementation
4 // always call rangeUpdate with true on tag
5 struct segtree {
6     struct node {
7         struct tag {
8             int val = 0;
9             bool has = false;
10        };
11        Hash hash;
12        int len = 0;
13        tag lazy;
14        static node combine(const node &a, const node &b){
15            if (a.len == 0) return b;
16            if (b.len == 0) return a;
17            extend(a.hash.len);
18            auto h1 = a.hash.h1 + pmod1[a.hash.len]*b.hash.h1;
19            auto h2 = a.hash.h2 + pmod2[a.hash.len]*b.hash.h2;
20            return {
21                {h1,h2,a.hash.len+b.hash.len},
22                a.len+b.len,
23                {0,false}
24            };
25        }
26        void apply(tag t){
27            if (!t.has) return;
28            // set hash as (t.val-offset)*sum
29            if (hash.len > 0){
30                extend(hash.len-1);
31                hash.h1 = (t.val-OFFSET)*presumpmod1[hash.len-1];
32                hash.h2 = (t.val-OFFSET)*presumpmod2[hash.len-1];
33            }

```

```

34     lazy = t;
35 }
36 void push(node &left, node &right){
37     if (lazy.has){
38         left.apply(lazy);
39         right.apply(lazy);
40         lazy = {0,false};
41     }
42 }
43 void pull(const node &left, const node &right){
44     tag cur = lazy;
45     *this = combine(left,right);
46     apply(cur);
47 }
48 };
49 int n, h;
50 vector<node> t;
51 segtree(const string& a) {
52     int sz = a.size();
53     n = 1; h = 0;
54     while (n < sz) { n*=2; h++; }
55
56     t.assign(2*n, node());
57
58     for (int i = 0; i < sz; i++) {
59         t[i+n].len = 1;
60         t[i+n].hash.len = 1;
61         t[i+n].apply({a[i], true});
62     }
63     for (int i = n-1; i > 0; i--) t[i].pull(t[i<<1], t[
64         i<<1|1]);
65 }
66 void build(int p) {
67     while (p > 1) p >>= 1, t[p].pull(t[p<<1], t[p
68         <<1|1]);
69 }
70 void push(int p) {
71     for (int s = h; s > 0; s--) {
72         int i = p >> s;
73         t[i].push(t[i<<1], t[i<<1|1]);
74     }
75 }
76 void rangeUpdate(int l, int r, node::tag v) {
77     if (l==r) return;
78     l += n, r += n;
79     int l0 = l, r0 = r;
80
81     push(l0); push(r0-1);
82     for (; l < r; l >>= 1, r >>= 1) {
83         if (l&1) t[l++].apply(v);
84         if (r&1) t[--r].apply(v);
85     }
86     build(l0); build(r0-1);
87 }
88 node query(int l, int r) {
89     if (l == r) return node();
90     l += n, r += n;
91     push(l); push(r-1);
92     node resL = node(), resR = node();
93     for (; l < r; l >>= 1, r >>= 1) {
94         if (l&1) resL = node::combine(resL, t[l++]);
95         if (r&1) resR = node::combine(t[--r], resR);
96     }
97     return node::combine(resL, resR);
98 }
99 };

```

2.8.3 Recursive Lazy

Best for: Index-dependent updates (e.g. AP addition), Tree Walking (Binary Search on Tree), and persistency.

- **Performance:** It is the slowest of the three, use only if necessary.

```

1 // LAZY recursive - implemented for sum + range AP
2 // addition
3 struct segtree {
4     struct node {
5         struct tag {
6             int a = 0, d = 0;
7             bool has = false;
8         };
9         int sum = 0; // data
10        tag lazy;
11        static node combine(const node& left, const node&
12            right) {
13            return {left.sum + right.sum, {0, 0, false}};
14        }
15        void leaf(int v) { sum = v; }
16        void apply(tag t, int lx, int rx) {
17            if (!t.has) return;
18            int len = rx - lx;
19
20            // Sum of (t.a + t.d * i) for i in [lx, rx-1]
21            sum += t.a * len + t.d*len*(lx+rx-1)/2;
22
23            lazy.a += t.a;
24            lazy.d += t.d;
25            lazy.has = true;
26        }
27        void push(node& left, node& right, int lx, int rx)
28        {
29            if (lazy.has) {
30                int mx = (lx + rx) / 2;
31                left.apply(lazy, lx, mx);
32                right.apply(lazy, mx, rx);
33                lazy = {0, 0, false};
34            }
35        }
36    };
37    int size;
38    vector<node> t;
39
40    void init(int n) {
41        size = 1;
42        while (size < n) size *= 2;
43        t.assign(2*size, node());
44        build(0, 0, size, n);
45    }
46    void build(int x, int lx, int rx, int n) {
47        if (rx-lx == 1) {
48            if (lx < n) t[x].leaf(0);
49            return;
50        }
51        int mx = (lx + rx) / 2;
52        build(2*x+1, lx, mx, n);
53        build(2*x+2, mx, rx, n);
54        t[x] = node::combine(t[2*x+1], t[2*x+2]);
55    }
56    void push(int x, int lx, int rx) {
57        t[x].push(t[2*x+1], t[2*x+2], lx, rx);
58    }
59    void rangeUpdate(int l, int r, node::tag v, int x,
60        int lx, int rx) {

```

```

58     if (lx >= r || rx <= l) return;
59     if (lx >= l && rx <= r) {
60         t[x].apply(v, lx, rx);
61         return;
62     }
63     push(x, lx, rx);
64     int mx = (lx + rx) / 2;
65     rangeUpdate(l, r, v, 2*x+1, lx, mx);
66     rangeUpdate(l, r, v, 2*x+2, mx, rx);
67     t[x] = node::combine(t[2*x+1], t[2*x+2]);
68 }
69 void rangeUpdate(int l, int r, int a, int d) {
70     rangeUpdate(l, r, {a - l*d, d, true}, 0, 0, size);
71 }
72 node query(int l, int r, int x, int lx, int rx) {
73     if (lx >= r || rx <= l) return node();
74     if (lx >= l && rx <= r) return t[x];
75     push(x, lx, rx);
76     int mx = (lx+rx) / 2;
77     return node::combine(
78         query(l, r, 2*x+1, lx, mx),
79         query(l, r, 2*x+2, mx, rx)
80     );
81 }
82 node query(int l, int r) {
83     return query(l, r, 0, 0, size);
84 }
85 };

```

2.9 Sparse Table RMQ

Sparse table for RMQ in $\mathcal{O}(1)$, used in many problems, including $\mathcal{O}(1)$ LCA (Trees) and LCP (SuffixArray) queries.

```

1 struct SparseTable {
2     vector<vector<ii>> st;
3
4     void build(const vi &a) {
5         int n = a.size();
6         int max_log = _lg(n)+1;
7         st.assign(max_log, vector<ii>(n));
8         for (int i = 0; i < n; i++) {
9             st[0][i] = {a[i], i};
10        }
11        for (int i = 1; i < max_log; i++) {
12            for (int j = 0; j + (1 << i) <= n; j++) {
13                // Combine the two halves
14                st[i][j] = f(st[i-1][j], st[i-1][j + (1 << i) - 1]);
15            }
16        }
17    }
18
19    // Returns min value and index in range [l, r]
20    // inclusive
21    ii min(int l, int r) {
22        int len = r - l + 1;
23        int k = _lg(len);
24        return f(st[k][l], st[k][r - (1<<k) + 1]);
25    }
26
27    inline ii f(ii l, ii r) { return std::min(l,r); }
28 };

```

3 Graphs

3.1 BFS 0-1

Time: $\mathcal{O}(n + m)$

```

1 vi bfs01(int s){
2   vi d(n, INF);
3   d[s] = 0;
4   deque<int> q;
5   q.push_front(s);
6   while(!q.empty()){
7     int u = q.front(); q.pop_front();
8     for (auto [w,v] : adj[u]){
9       if (d[u]+w < d[v]){
10        d[v] = d[u] + w;
11        if (w == 1) q.push_back(v);
12        else q.push_front(v);
13      }
14    }
15  }
16  return d;
17 }
```

3.2 Block Cut Tree (BCT)

A data structure that condenses an undirected graph into a bipartite graph using two types of nodes: B-nodes (biconnected components) and C-nodes (articulation points). Edges strictly connect a B-node to a C-node if the articulation point belongs to that component. Yields exactly one tree per connected component.

Articulation points are explicitly represented as the C-nodes. Because an articulation point is a vertex that disconnects a component upon removal, it acts as a literal bridge in the BCT. Any path navigating between two distinct biconnected components in the original graph is forced to route through the shared C-node.

```

1 vvi build_bct(const vvi &adj){
2   int n = adj.size();
3   vi in(n,-1), low(n), st;
4   int timer = 0;
5   vvi bccs;
6
7   auto dfs = [&](auto &&dfs, int u, int p) -> void {
8     in[u] = low[u] = ++timer;
9     st.push_back(u);
10    if (p == -1 && adj[u].empty()){
11      bccs.push_back({u});
12      st.pop_back();
13      return;
14    }
15    for (int v : adj[u]) if (v != p) {
16      if (in[v] != -1) {
17        low[u] = min(low[u], in[v]);
18      } else {
19        dfs(dfs,v,u);
20        low[u] = min(low[u],low[v]);
21        if (low[v] >= in[u]){
```

```

22      bccs.push_back({u});
23      while(1){
24        int w = st.back();
25        st.pop_back();
26        bccs.back().push_back(w);
27        if (w == v) break;
28      }
29    }
30  }
31  }
32  };
33  for (int i = 0; i < n; i++){
34    if (in[i] == -1) dfs(dfs,i,-1);
35  }
36  vvi bct(n+bccs.size());
37  for (int i = 0; i < bccs.size(); i++){
38    int b = n+i;
39    for (int u : bccs[i]){
40      bct[u].push_back(b);
41      bct[b].push_back(u);
42    }
43  }
44  return bct;
45 }
```

3.3 Bridges and Articulation points

DFS to get bridges and articulation points of a graph in $\mathcal{O}(n + m)$

```

1 vvi adj;
2 vi in, low;
3 int timer;
4 set<int> cut_points;
5 vector<ii> bridges;
6
7 void dfs_ap(int u, int p = -1) {
8   in[u] = low[u] = ++timer;
9   int ch = 0;
10  for (int v : adj[u]) {
11    if (v == p) continue;
12    if (in[v]) {
13      // Back-edge
14      low[u] = min(low[u], in[v]);
15    } else {
16      // Tree-edge
17      dfs_ap(v, u);
18      low[u] = min(low[u], low[v]);
19      if (low[v] >= in[u] && p != -1)
20        cut_points.insert(u);
21      ch++;
22    }
23  }
24  if (p == -1 && ch > 1)
25    cut_points.insert(u);
26 }
27
28 void dfs_bridges(int u, int p = -1) {
29   in[u] = low[u] = ++timer;
30   for (int v : adj[u]) {
31     if (v == p) continue;
32     if (in[v]) {
33       low[u] = min(low[u], in[v]);
34     } else {
35       dfs_bridges(v, u);
36       low[u] = min(low[u], low[v]);
37       if (low[v] > in[u])
```

```

38     bridges.push_back({u, v});
39   }
40 }
41 }
42
43 void init(int n) {
44   timer = 0;
45   in.assign(n, 0);
46   low.assign(n, 0);
47   cut_points.clear();
48   bridges.clear();
49 }
```

3.4 Dijkstra

Time: $\mathcal{O}(m \log n)$

```

1 void dijkstra(int s){
2   int d, u, v;
3   dist = vi(n, INF);
4   dist[s] = 0;
5   priority_queue<ii, vii, greater<ii>> pq;
6   pq.emplace(0,s);
7   while(!pq.empty()){
8     auto [d,u] = pq.top(); pq.pop();
9     if (d > dist[u]) continue;
10
11     for (auto &[w,v] : adj[u]){
12       if (dist[v] > dist[u] + w){
13         dist[v] = dist[u] + w;
14         pq.emplace(dist[v], v);
15       }
16     }
17   }
18 }
```

3.5 Dinic - Flow/matchings

- **General Network:** $\mathcal{O}(VE \log U)$.
- **Unit Capacity Network:** $\mathcal{O}(\min(V^{2/3}, E^{1/2}) \cdot E)$. Often considered $\mathcal{O}(E\sqrt{V})$.
- **Bipartite Matching:** $\mathcal{O}(\min(V^{2/3}, E^{1/2}) \cdot E)$. Often considered $\mathcal{O}(E\sqrt{V})$.

```

1 struct Dinic {
2   struct Edge {
3     int u, v;
4     ll cap, flow = 0;
5     Edge(int u, int v, ll cap) : u(u), v(v), cap(cap) {}
6   };
7
8   const ll flow_inf = 1e18;
9   vector<Edge> edges;
10  vvi adj;
11  int n, m = 0;
12  int s, t;
13  vi level, ptr;
14  queue<int> q;
15
16  Dinic(int n): n(n) {
```

```

17     adj.resize(n);
18     level.resize(n);
19     ptr.resize(n);
20 }
21
22 void add_edge(int u, int v, ll cap) {
23     edges.emplace_back(u,v,cap);
24     edges.emplace_back(v,u,0);
25     adj[u].push_back(m++);
26     adj[v].push_back(m++);
27 }
28
29 bool bfs(ll delta){
30     queue<int> q;
31     q.push(s);
32     while(!q.empty()){
33         int u = q.front(); q.pop();
34         for (int id : adj[u]){
35             auto &e = edges[id];
36             if (e.cap - e.flow < delta) continue;
37             if (level[e.v] != -1) continue;
38             level[e.v] = level[u]+1;
39             q.push(e.v);
40         }
41     }
42     return level[t] != -1;
43 }
44
45 ll dfs(int u, ll pushed) {
46     if (pushed == 0) return 0;
47     if (u == t) return pushed;
48     for (int &cid = ptr[u]; cid < (int)adj[u].size(); cid++){
49         int id = adj[u][cid];
50         auto &e = edges[id];
51         if (level[u]+1 != level[e.v]) continue;
52         ll tr = dfs(e.v,min(pushed, e.cap - e.flow));
53         if (tr == 0) continue;
54         e.flow += tr;
55         edges[id^1].flow -= tr;
56         return tr;
57     }
58     return 0;
59 }
60
61 ll maxflow(int s, int t){
62     this->s = s; this->t = t;
63     ll max_c = 0;
64     for (auto &e : edges) max_c = max(max_c, e.cap);
65
66     ll delta = 1;
67     while(delta <= max_c) delta <<= 1;
68     delta >>= 1;
69
70     ll f = 0;
71     for (;delta > 0; delta >>= 1){
72         while(true){
73             fill(level.begin(), level.end(),-1);
74             level[s] = 0;
75             if (!bfs(delta)) break;
76             fill(ptr.begin(), ptr.end(), 0);
77             while(ll pushed = dfs(s,flow_inf)) f += pushed;
78         }
79     }
80     return f;
81 }
82
83 // call constructor with (n1+n2+2) beforehand (dont
84 // assumes pairs are 1-indexed

```

```

85     vii maxmatchings(int n1, int n2, const vii& pairs){
86         for (int i = 1; i <= n1; i++){
87             add_edge(0,i,1);
88
89             for (int i = 1; i <= n2; i++){
90                 add_edge(i+n1,n-1,1);
91
92                 for (auto &[u,v] : pairs)
93                     add_edge(u,v+n1,1);
94
95                 maxflow(0,n-1);
96
97                 vii matchings;
98                 for (auto &e : edges){
99                     if (e.u >= 1 && e.u <= n1 && e.flow == 1 && e.v >
100                        n1){
101                         matchings.emplace_back(e.u,e.v-n1);
102                     }
103                 }
104                 return matchings;
105             }
106         }
107
108         vii mincut(int s, int t){
109             maxflow(s,t);
110             queue<int> q; q.push(s);
111             vector<bool> reachable(n);
112             reachable[s] = true;
113             while(!q.empty()){
114                 int u = q.front(); q.pop();
115                 for (auto &id : adj[u]){
116                     int v = edges[id].v;
117                     if (edges[id].cap - edges[id].flow > 0 && !
118                         reachable[v]) {
119                         reachable[v] = true;
120                         q.push(v);
121                     }
122                 }
123             }
124
125             vii minCutEdges;
126
127             for (int i = 0; i < m; i += 2) {
128                 const Edge& edge = edges[i];
129                 if (reachable[edge.u] && !reachable[edge.v]) {
130                     minCutEdges.emplace_back(edge.u, edge.v);
131                 }
132             }
133             return minCutEdges;
134         };

```

3.6 Floyd-Warshall

Time: $\mathcal{O}(n^3)$

```

1     vvi d(n, vi(n, INF));
2     void floyd_warshall(){
3         for (int k = 0; k < n; k++){
4             for (int i = 0; i < n; i++){
5                 for (int j = 0; j < n; j++){
6                     d[i][j] = min(d[i][j], d[i][k]+d[k][j]);
7                 }
8             }

```

3.7 Hopcroft-Karp - Bipartite Matching

Bipartite matching such as Kuhn but faster. BFS until first layer missing match, DFS for the BFS graph to find pairings. Time: $\mathcal{O}(E\sqrt{V})$

```

1     int n, m, k;
2     vvi adj;
3     vi p, dist; /*p is in matching for [0, n[ and parent
4                  for [n, n+m[*/
5
6     int bfs(){
7         queue<int> q;
8         dist = vi(n+m, inf);
9         for(int i = 0; i < n; i++){
10             if(p[i] == -1) q.push(i), dist[i] = 0;
11         }
12         int min_dist_match = inf;
13         while(!q.empty()){
14             int u = q.front(); q.pop();
15             if(dist[u] > min_dist_match) continue;
16             for(auto v: adj[u]){
17                 if(p[v] == -1) min_dist_match = dist[u];
18                 else if(dist[p[v]] == inf){
19                     dist[p[v]] = dist[u] + 1;
20                     q.push(p[v]);
21                 }
22             }
23         }
24         return min_dist_match != inf;
25
26     int dfs(int u){
27         for(auto v: adj[u]){
28             if(p[v] == -1 || (dist[u]+1 == dist[p[v]] && dfs(p[
29                 v]))){
30                 p[v] = u;
31                 p[u] = 1;
32                 return true;
33             }
34         }
35         dist[u] = inf;
36         return false;
37     }
38
39     int hopkarp(){
40         p = vi(n+m, -1);
41         int matchings = 0;
42         while(bfs()){
43             for(int i = 0; i < n; i++){
44                 if(p[i] == -1 && dfs(i)) matchings++;
45             }
46         }
47         return matchings;
48
49     void create(){
50         adj = vvi(n+m);
51         for(int i = 0; i < k; i++){
52             int u, v;
53             cin >> u >> v; u--; v--;
54             v += n;
55             adj[u].push_back(v);
56         }
57     }

```

3.8 Hungarian

Solves minimum cost assignment for n workers and m jobs.

Time: $\mathcal{O}((n+m)^3)$

```

1 // cost should be (cost[worker][job])
2 pair<int,vii> hungarian(int n, int m, const vvi &cost)
3 {
4     if (n == 0) return {0,{} };
5     int N = max(n, m);
6
7     vi u(N+1), v(N+1), p(N+1), way(N+1);
8
9     const int INF = 1e9;
10    for (int i = 1; i <= n; ++i) {
11        p[0] = i;
12        int j0 = 0;
13        vi minv(N+1, INF);
14        vector<bool> used(N+1, false);
15
16        do {
17            used[j0] = true;
18            int i0 = p[j0], delta = INF, j1;
19
20            for (int j = 1; j <= N; ++j) {
21                if (!used[j]) {
22                    int cur = cost[i0-1][j-1] - u[i0] - v[j];
23                    if (cur < minv[j]) {
24                        minv[j] = cur;
25                        way[j] = j0;
26                    }
27                }
28                if (minv[j] < delta) {
29                    delta = minv[j];
30                    j1 = j;
31                }
32            }
33
34            for (int j = 0; j <= N; ++j) {
35                if (used[j]) {
36                    u[p[j]] += delta;
37                    v[j] -= delta;
38                } else {
39                    minv[j] -= delta;
40                }
41            }
42            j0 = j1;
43        } while (p[j0] != 0);
44
45        do {
46            int j1 = way[j0];
47            p[j0] = p[j1];
48            j0 = j1;
49        } while (j0);
50    }
51
52    int total_cost = 0;
53    for (int j = 1; j <= m; ++j) {
54        if (p[j] != 0) {
55            total_cost += cost[p[j]-1][j-1];
56        }
57    }
58
59    // {worker, job}[] 0-indexed
60    vii matchings;
61    for (int j = 1; j <= m; ++j) {
62        if (p[j] != 0) {
63            matchings.push_back({p[j]-1, j-1});
64        }
65    }
66    return {total_cost, matchings};
67 }

```

3.9 Kuhn - Bipartite Matching

Bipartite matching. Time: $\mathcal{O}(VE)$

```

1 int matchings;
2 vi p, vis;
3 vii match;
4
5 int dfs(int u){
6     if(vis[u]) return 0;
7     vis[u] = 1;
8     for(auto v: adj[u]){
9         if(p[v] == -1 || dfs(p[v])){
10             p[v] = u;
11             return 1;
12         }
13     }
14     return 0;
15 }
16
17 void kuhn(){
18     matchings = 0;
19     p = vi(n+m, -1);
20     for(int i = 0; i < n; ++i){
21         vis = vi(n, 0);
22         matchings += dfs(i);
23     }
24     for(int i = n; i < n+m; i++){
25         if(p[i] != -1) match.push_back(ii(p[i], i));
26     }
27 }
28
29 void create(){
30     adj = vvi(n+m);
31     for(int i = 0; i < k; ++i){
32         int u, v;
33         cin >> u >> v; u--; v--;
34         adj[u].push_back(v+n);
35     }
36 }

```

3.10 Min cost flow

Time: $\mathcal{O}(FE \log V)$

If negative costs are needed (maximize cost), need to run SPFA once at the start, making the solution $\mathcal{O}(EV + FE \log V)$.

```

1 struct MinCostFlow {
2     struct Edge {
3         int to, capacity, rev;
4         ll cost;
5     };
6
7     int n;
8     vector<vector<Edge>> adj;
9
10    MinCostFlow(int _n): n(_n), adj(_n) {}
11 }

```

```

12 void add_edge(int from, int to, int cap, ll cost){
13     adj[from].push_back({to, cap, (int)adj[to].size(),
14         cost});
15     adj[to].push_back({from, 0, (int)adj[from].size()-1,
16         -cost});
17 }
18
19 // 0(FE log(V))
20 lli min_cost_flow(int s, int t, int targetFlow) {
21     int flow = 0;
22     ll total_cost = 0;
23     vll dist, h(n);
24     vi pv, pe;
25
26     // needed only if negative costs exists
27     spfa(s, h, pv, pe);
28
29     while (flow < targetFlow) {
30         dijkstra(s, h, dist, pv, pe);
31
32         if (dist[t] == INF) break;
33
34         for (int i = 0; i < n; i++) {
35             if (dist[i] < INF) {
36                 h[i] += dist[i];
37             }
38         }
39
40         int f = targetFlow - flow;
41         int cur = t;
42         while (cur != s) {
43             f = min(f, adj[pv[cur]][pe[cur]].capacity);
44             cur = pv[cur];
45         }
46
47         flow += f;
48         total_cost += f * h[t];
49         cur = t;
50         while (cur != s) {
51             Edge &e = adj[pv[cur]][pe[cur]];
52             e.capacity -= f;
53             adj[e.to][e.rev].capacity += f;
54             cur = pv[cur];
55         }
56
57         return {total_cost, flow};
58     }
59
60     // needed only if negative costs exists
61     void spfa(int s, vll &dist, vi &pv, vi &pe) {
62         dist.assign(n, INF);
63         pv.assign(n, -1);
64         pe.assign(n, -1);
65         vector<bool> inq(n, false);
66         queue<int> q;
67
68         dist[s] = 0;
69         q.push(s);
70         inq[s] = true;
71
72         while (!q.empty()) {
73             int u = q.front(); q.pop();
74             inq[u] = false;
75             for (int i = 0; i < adj[u].size(); i++) {
76                 Edge &e = adj[u][i];
77                 int v = e.to;
78                 if (e.capacity > 0 && dist[v] > dist[u] + e.
79                     cost) {

```

```

79         dist[v] = dist[u] + e.cost;
80         pv[v] = u;
81         pe[v] = i;
82         if (!inq[v]) {
83             inq[v] = true;
84             q.push(v);
85         }
86     }
87 }
88 }
89 }
90
91 void dijkstra(int s, vll &h, vll &dist, vi &pv, vi &
92             pe) {
93     dist.assign(n, INF);
94     pv.assign(n, -1);
95     pe.assign(n, -1);
96     dist[s] = 0;
97
98     priority_queue<lli, vector<lli>, greater<lli>> pq;
99     pq.emplace(0, s);
100
101     while (!pq.empty()) {
102         auto [d, u] = pq.top(); pq.pop();
103         if (d > dist[u]) continue;
104
105         for (int i = 0; i < adj[u].size(); i++) {
106             Edge &e = adj[u][i];
107             if (e.capacity <= 0) continue;
108             int v = e.to;
109
110             ll reduced_cost = e.cost + h[u] - h[v];
111             if (dist[u] != INF && dist[v] > dist[u] +
112                 reduced_cost) {
113                 dist[v] = dist[u] + reduced_cost;
114                 pv[v] = u;
115                 pe[v] = i;
116                 pq.push({dist[v], v});
117             }
118         }
119     };
120
121     // usage
122     int nodes = 302; // amount of nodes in the network
123     MinCostFlow mcf(nodes);
124
125     for (int i = 0; i < 150; i++){
126         mcf.add_edge(0, i+1, 1, 0); // source to node
127         mcf.add_edge(i+151, nodes-1, 1, 0); // node to sink
128     }
129
130     for (int i = 0; i < n; i++){
131         int a, b, c; cin >> a >> b >> c;
132         mcf.add_edge(a, b+150, 1, -c); // edges in between (-
133             c to maximize the cost)
134     }
135
136     // final max cost is -cost
137     auto [cost, flow] = mcf.min_cost_flow(0, nodes-1, 150);

```

3.11 MST - Kruskal

Time: $\mathcal{O}(m \log m)$

```

1 vector<pair<int,ii>> edges; // [weight, (u,v)]
2 int kruskal(int n){

```

```

3     int cost = 0;
4     DSU dsu(n); // n is the numb of vertices
5     sort(edges.begin(), edges.end());
6     for (auto &[w,uv] : edges){
7         auto [u,v] = uv;
8         if (dsu.join(u,v)) cost += w;
9     }
10    return cost;
11 }

```

3.12 MST - Prim

Time: $\mathcal{O}(m \log n)$

```

1 vvi adj, mst;
2 vi taken;
3
4 int prim(){
5     priority_queue<iii, vector<iii>, greater<iii>> pq;
6     taken[0] = 1;
7     for (auto [w,v] : adj[0]){
8         if (!taken[v]) pq.push({w, {0,v}});
9     }
10
11     int cost = 0;
12     while (!pq.empty()){
13         auto [w,pu] = pq.top(); pq.pop();
14         auto [p,u] = pu;
15         if (!taken[u]) {
16             cost += w;
17             mst[p].emplace_back(w,u);
18             mst[u].emplace_back(w,p);
19             taken[u] = 1;
20             for (auto [w,v] : adj[u]){
21                 if (!taken[v]) {
22                     pq.push({w,{u,v}});
23                 }
24             }
25         }
26     }
27     return cost;
28 }

```

3.13 SCC compressing

Condensing a graph into a DAG through its strongly connected components can be useful for DP

```

1 struct SCCCondenser {
2     int n, timer, scc_cnt;
3     vi in, low, scc;
4     stack<int> st;
5
6     SCCCondenser(const vvi& adj) {
7         n = adj.size();
8         in.assign(n, 0);
9         low.assign(n, 0);
10        scc.assign(n, -1);
11        timer = scc_cnt = 0;
12        for (int i = 0; i < n; ++i)
13            if (!in[i]) dfs(i, adj);
14    }
15
16    void dfs(int u, const vvi& adj) {
17        in[u] = low[u] = ++timer;

```

```

18        st.push(u);
19        for (int v : adj[u]) {
20            if (!in[v]) {
21                dfs(v, adj);
22                low[u] = min(low[u], low[v]);
23            } else if (scc[v] == -1) {
24                low[u] = min(low[u], in[v]);
25            }
26        }
27        if (low[u] == in[u]) {
28            while (true) {
29                int v = st.top(); st.pop();
30                scc[v] = scc_cnt;
31                if (u == v) break;
32            }
33            scc_cnt++;
34        }
35    }
36
37    // Returns {DAG, Aggregated Values}
38    pair<vvi, vi> build(const vvi& adj, const vi& val) {
39        vvi dag(scc_cnt);
40        vi scc_val(scc_cnt);
41        set<ii> edges;
42
43        for (int u = 0; u < n; ++u) {
44            scc_val[scc[u]] += val[u]; // Aggregate values
45            for (int v : adj[u]) {
46                if (scc[u] != scc[v]) {
47                    if (edges.count({scc[u], scc[v]})) continue;
48                    edges.insert({scc[u], scc[v]});
49                    dag[scc[u]].push_back(scc[v]);
50                }
51            }
52        }
53        return {dag, scc_val};
54    }
55 };

```

4 DP

4.1 Bin Packing

Time: $\mathcal{O}(n \cdot 2^n)$ Space: $\mathcal{O}(2^n)$

```

1 vi w(n);
2
3 vector<ii> dp(1<<n, ii(INF,0));
4 // dp[i] = for the subset i(bitmask) (A,B) is the pair
5     where
6 // A - the min number of knapsacks to store this subset
7 // B - the min size of a used knapsack
8
9 dp[0] = ii(0,INF);
10 for (int subset = 1; subset < (1<<n); subset++){
11     for (int item = 0; item < n; item++){
12         if (!((subset>>item)&1)) continue;
13         int prevsubset = subset - (1<<item);
14         ii prev = dp[prevsubset];
15
16         if (prev.second + w[item] <= x) {
17             // can fill the knapsack, fill it
18             dp[subset] = min(dp[subset], ii(prev.first, prev.
19                 second+w[item]));

```

```

20     dp[subset] = min(dp[subset], ii(prev.first+1, w[
        item]));
21 }
22 }
23 }
24
25 cout << dp[(1<<n)-1].first << endl;

```

4.2 Broken Profile DP

Solves the problem of counting how many ways to fill an $n \times m$ grid using 1×2 tiles. This technique can be used whenever the state dependence is only on the previous state (column). Time: $\mathcal{O}(mn2^n)$ Space: $\mathcal{O}(mn2^n)$

```

1 int dp[1002][12][1024];
2 dp[0][0][0] = 1;
3
4 for (int i = 0; i < m; i++){
5     for (int j = 0; j < n; j++){
6         for (int mask = 0; mask < (1<<n); mask++){
7             if (mask & (1<<j)){
8                 int nxt_mask = mask - (1<<j);
9                 dp[i][j+1][nxt_mask] += dp[i][j][mask];
10                dp[i][j+1][nxt_mask] %= M;
11            } else {
12                int q = mask + (1 << j);
13                dp[i][j+1][q] += dp[i][j][mask];
14                dp[i][j+1][q] %= M;
15                if (j < n-1 && (mask & (1<<(j+1)))==0){
16                    q = mask + (1 << (j+1));
17                    dp[i][j+1][q] += dp[i][j][mask];
18                    dp[i][j+1][q] %= M;
19                }
20            }
21        }
22    }
23 }
24 for (int p = 0; p < (1<<n); p++){
25     dp[i+1][0][p] = dp[i][n][p];
26 }
27 }

```

4.3 Convex Hull Trick (CHT)

• Recurrence form:

TODO formulas

- **Slope monotonicity:** If coefficients a_j (slopes) are inserted in strictly decreasing (or increasing) order as j grows, and
- **Query monotonicity:** Values x_i for query come in non-decreasing (min) or increasing (max) order consistent with slope order,
- **Complexity:**

- Insertion + amortized query in $\mathcal{O}(1)$ per operation (pointer walk) under monotonicity.
- Non-monotonic case, generic CHT via binary search: $\mathcal{O}(\log n)$ per query.
- General alternative: Li Chao Tree for insertion/s/queries in arbitrary order, $\mathcal{O}(\log M)$ per operation (where M is the domain of x).

• Constraints:

- If it cannot be written in linear form, CHT does not apply.
- If there is no monotonicity of slopes or queries, consider Li Chao Tree or CHT variant with binary search.

The example below solves the dp where the recurrence is:

TODO formulas

```

1 struct CHT {
2     struct Line { // y = mx + c
3         int m, c;
4         Line(int m, int c) : m(m), c(c) {}
5         int val(int x){
6             return m*x + c;
7         }
8         int floorDiv(int num, int den) {
9             if (den < 0) num = -num, den = -den;
10            if (num >= 0) return num / den;
11            else return - ( (-num + den - 1) / den );
12        }
13        int ceilDiv(int num, int den) {
14            if (den < 0) num = -num, den = -den;
15            if (num >= 0) return (num + den - 1) / den;
16            else return - ( (-num) / den );
17        }
18        int intersect(Line l){
19            // m1x + c1 = m2x + c2
20            // x = (c2 - c1)/(m1 - m2)
21            // if slopes are increasing, use floor div
22            return ceilDiv(l.c - c, m - l.m);
23        }
24    };
25
26    deque<pair<Line, int>> dq;
27
28    void insert(int m, int c){
29        Line newLine(m, c);
30        if (!dq.empty() && newLine.m == dq.back().first.m)
31            {
32                // If slopes increasing, change to <=
33                if (newLine.c >= dq.back().first.c) return;
34                else dq.pop_back();
35            }
36        // if slopes increasing, change to <=
37        while (dq.size() > 1 && dq.back().second >= dq.back()
38            ().first.intersect(newLine)){
39            dq.pop_back();
40        }
41        if (dq.empty()){
42            // assuming queries are positive numbers, may
43            // change to -INF or +INF if needed
44            dq.emplace_back(newLine, 0);

```

```

42         return;
43     }
44     dq.emplace_back(newLine, dq.back().first.intersect(
45         newLine));
46 }
47 // dont use query and queryNonMonotonicValues in the
48 // same problem
49 int query(int x){
50     while (dq.size() > 1){
51         // if slopes increasing, change to >=
52         if (dq[1].second <= x) dq.pop_front();
53         else break;
54     }
55     return dq[0].first.val(x);
56 }
57
58 int queryNonMonotonicValues(int x){
59     int l=0, r=dq.size()-1, ans=0;
60     while (l <= r) {
61         int mid = (l+r)>>1;
62         if (dq[mid].second <= x) {
63             ans = mid;
64             l = mid + 1;
65         } else {
66             r = mid - 1;
67         }
68     }
69     return dq[ans].first.val(x);
70 }
71 };
72
73 void solve(){
74     int n, c; cin >> n >> c;
75     vi h(n);
76     for (auto &x : h) cin >> x;
77
78     vi dp(n);
79     dp[0] = 0;
80     CHT cht;
81     cht.insert(-2*h[0], h[0]*h[0]);
82     for (int i = 1; i < n; i++){
83         dp[i] = cht.query(h[i]) + c + h[i]*h[i];
84         cht.insert(-2*h[i], h[i]*h[i] + dp[i]);
85     }
86     cout << dp[n-1] << endl;
87 }

```

4.4 Edit Distance (Levenshtein)

Very similar to LCS, in the sense that it considers prefixes already computed. Time: $\mathcal{O}(mn)$ Space: $\mathcal{O}(mn)$

```

1 vvi dp(n+1, vi(m+1));
2 for (int i = 0; i <= n; i++) dp[i][0] = i;
3 for (int i = 0; i <= m; i++) dp[0][i] = i;
4 for (int i = 1; i <= n; i++){
5     for (int j = 1; j <= m; j++){
6         dp[i][j] = min(
7             min(dp[i][j-1]+1, dp[i-1][j]+1),
8             dp[i-1][j-1] + (s[i-1]!=t[j-1])
9         );
10    }
11 }

```

4.5 Knapsack - 1D

The spirit here is the same as the 2D version, but here it iterates on the knapsack capacity backwards, to ensure that the value of $dp[j-w[i]]$ is not considering the item i . Time: $\mathcal{O}(nW)$ Space: $\mathcal{O}(W)$

```
1 vi dp(W+1);
2 for (int i = 0; i < n; i++){
3     for (int j = W; j >= w[i]; j--){
4         dp[j] = max(dp[j], v[i] + dp[j-w[i]]);
5     }
6 }
```

4.6 Knapsack - 2D

Time: $\mathcal{O}(nW)$ Space: $\mathcal{O}(nW)$

```
1 vvi dp(n+1,vi(W+1));
2 for (int c = 1; c <= W; c++){
3     for (int i = 1; i <= n; i++){
4         dp[i][c] = dp[i-1][w];
5         if (c-w[i-1] >= 0) {
6             dp[i][c] = max(dp[i][c], dp[i-1][c-w[i-1]] + v[i-1]);
7         }
8     }
9 }
```

4.7 LCS - Longest Common Subsequence

Subsequence generation included here. Time: $\mathcal{O}(mn)$ Space: $\mathcal{O}(mn)$

```
1 vvi dp(n+1,vi(m+1));
2
3 for (int i = 1; i <= n; i++){
4     for (int j = 1; j <= m; j++){
5         if (a[i-1] == b[j-1]) {
6             dp[i][j] = dp[i-1][j-1]+1;
7         } else if (dp[i][j-1] > dp[i-1][j]){
8             dp[i][j] = dp[i][j-1];
9         } else {
10            dp[i][j] = dp[i-1][j];
11        }
12    }
13 }
```

4.8 LiChao Tree

Generalization of CHT for linear functions that do not need to be sorted. Essentially a sparse segtree. Queries and insertions are all $\mathcal{O}(\log M)$. Where M is the size of the query interval the tree receives.

```
1 // Li Chao tree for minimum (or maximum) over domain [L, R].
2 // T should support +, *, comparisons.
```

```
3 // For integer x use eps = 0 and discrete mid+1 splitting;
4 // For floating use eps > 0 and continuous splitting without +1.
5 template<typename T>
6 struct lichao_tree {
7     // if max lichao, change to ::min()
8     static const T identity = numeric_limits<T>::max();
9
10    struct Line {
11        T m, c;
12        Line() {
13            m = 0;
14            c = identity;
15        }
16        Line(T m, T c) : m(m), c(c) {}
17        T val(T x) { return m * x + c; }
18    };
19
20    struct Node {
21        Line line;
22        Node *lc, *rc;
23        Node() : lc(0), rc(0) {}
24    };
25
26    T L, R, eps;
27    deque<Node> buffer;
28    Node* root;
29
30    Node* new_node() {
31        buffer.emplace_back();
32        return &buffer.back();
33    }
34
35    lichao_tree() {}
36
37    lichao_tree(T _L, T _R, T _eps) {
38        init(_L, _R, _eps);
39    }
40
41    void clear() {
42        buffer.clear();
43        root = nullptr;
44    }
45
46    void init(T _L, T _R, T _eps) {
47        clear();
48        L = _L;
49        R = _R;
50        eps = _eps;
51        root = new_node();
52    }
53
54    void insert(Node* &cur, T l, T r, Line line) {
55        if (!cur) {
56            cur = new_node();
57            cur->line = line;
58            return;
59        }
60
61        T mid = l + (r - l) / 2;
62        if (r - l <= eps) return;
63
64        // if max lichao, change to >
65        if (line.val(mid) < cur->line.val(mid))
66            swap(line, cur->line);
67
68        // if max lichao, change to >
69        if (line.val(l) < cur->line.val(l)) insert(cur->lc, l, mid, line);
```

```
70        else insert(cur->rc, mid + 1, r, line);
71    }
72
73    T query(Node* &cur, T l, T r, T x) {
74        if (!cur) return identity;
75
76        T mid = l + (r - l) / 2;
77        T res = cur->line.val(x);
78        if (r - l <= eps) return res;
79
80        // if max lichao, change min to max
81        if (x <= mid) return min(res, query(cur->lc, l, mid, x));
82        else return min(res, query(cur->rc, mid + 1, r, x));
83    }
84
85    void insert(T m, T c) { insert(root, L, R, Line(m, c)); }
86
87    T query(T x) { return query(root, L, R, x); }
88 }
```

4.9 LIS - Longest Increasing Subsequence

Time: $\mathcal{O}(n \log n)$

```
1 // dp[i] := min value a subsequence of length i+1 can end in
2 // dp is an increasing sequence, allowing for bin search
3 int lis(const vi &a){
4     vi dp;
5     for (int x : a){
6         auto it = lower_bound(all(dp), x);
7         if (it == dp.end()) dp.push_back(x);
8         else *it = x;
9     }
10    return dp.size();
11 }
```

4.10 SOSDP

```
1 int k; // amount of bits
2 vi a(1<<k);
3 // sosdp
4 for (int bit = 0; bit < k; bit++){
5     for (int mask = 0; mask < (1<<k); mask++){
6         if (mask>>bit&1) {
7             a[mask] += a[mask ^ (1<<bit)];
8         }
9     }
10 }
11
12 // do stuff (such as multiplication for OR convolution)
13
14 // sosdp inverse
15 for (int bit = 0; bit < k; bit++){
16     for (int mask = 0; mask < (1<<k); mask++){
17         if (mask>>bit&1) {
18             a[mask] -= a[mask ^ (1<<bit)];
19         }
20     }
21 }
```

4.11 Subset Sum

Almost identical to Knapsack, this code contains the subset reconstruction. Time: $\mathcal{O}(nS)$ Space: $\mathcal{O}(nS)$

```

1 vvi dp(n+1, vi(sum+1));
2 vvii p(n+1, vii(sum+1));
3
4 dp[0][0] = 1;
5
6 for (int i = 1; i <= n; i++){
7     for (int s = 1; s <= sum; s++){
8         if (s-a[i-1] >= 0 && dp[i-1][s-a[i-1]]){
9             // sum is possible taking item i
10            p[i][s] = {i-1, s-a[i-1]};
11            dp[i][s] = 1;
12        } else if (dp[i-1][s]) {
13            // sum not possible taking item i
14            // but still possible with other items (<i)
15            p[i][s] = {i-1, s};
16            dp[i][s] = 1;
17        }
18    }
19 }
20
21 if (!dp[n][target]) {
22     cout << -1 << endl;
23     return;
24 }
25
26 vi subset;
27 ii pos = {n, target};
28 while(pos != ii(0,0)){
29     auto [i, s] = pos;
30     if (p[i][s].second != s) subset.push_back(a[i-1]);
31     pos = p[i][s];
32 }

```

4.12 Subset Sum with Bitset

If you only need reachability: Time: $\mathcal{O}(nS/w)$ (where w is sys word size) Space: $\mathcal{O}(S)$

```

1 vi a;
2 bitset<MAXSUM+1> dp; dp[0] = 1;
3 for (int x : a) dp |= (dp << x);

```

Trees

5.1 Centroid Decomposition

Recursively splits a tree by its center of mass, building a "Centroid Tree" with a strictly guaranteed $\mathcal{O}(\log N)$ depth. Used for path queries where the tree is unrooted.

Build: $\mathcal{O}(N \log N)$

Sample is solving "count distinct paths with number of edges between k_1 and k_2 "

```

1 struct CentroidDecomposition{
2     vvi t, ct;
3     vi sz, added, cp;
4     int n, root;
5     CentroidDecomposition(const vvi &adj, int _k1, int
        _k2)
6     {
7         t = adj,
8         n = adj.size();
9         cp = sz = added = vi(n);
10        root = -1;
11        ct.resize(n);
12        init(_k1, _k2);
13        decompose(0, -1);
14    }
15    int dfs(int u, int p){
16        sz[u] = 1;
17        for (int v : t[u]){
18            if (v == p || added[v]) continue;
19            sz[u] += dfs(v, u);
20        }
21        return sz[u];
22    }
23    int centroid(int u, int size, int p = -1){
24        for (int v : t[u]){
25            if (v == p || added[v]) continue;
26            if (2*sz[v] > size) return centroid(v, size, u);
27        }
28        return u;
29    }
30    void decompose(int u, int p){
31        int size = dfs(u, p);
32        int c = centroid(u, size);
33        added[c] = 1;
34
35        if (p == -1) root = c;
36        else ct[p].push_back(c);
37        cp[c] = p;
38
39        process_centroid(c);
40
41        for (int v : t[c])
42            if (!added[v])
43                decompose(v, c);
44    }
45
46    // custom stuff
47    int k1, k2;
48    ll ans;
49    vi freq, distfreq;
50    void init(int _k1, int _k2){
51        ans = 0;
52        k1 = _k1; k2 = _k2;
53        distfreq = freq = vi(k2+1);
54        freq[0] = 1;
55    }
56    void process_centroid(int c){
57        int mxd = 0;
58        int validsum = 0;
59        for (int u : t[c]) if (!added[u]){
60            distfreq[0] = 1;
61            int mxd = 0;
62            auto getdist = [&](auto &&getdist, int u, int p,
                int d) -> void {
63                distfreq[d]++;
64                mxd = max(mxd, d);
65                if (d == k2) return;
66                for (int v : t[u]){
67                    if (v == p || added[v]) continue;
68                    getdist(getdist, v, u, d+1);

```

```

69        }
70    }; getdist(getdist, u, c, 1);
71    int sum = validsum;
72    for (int d = 1; d <= mxd; d++){
73        ans += 1ll*sum*distfreq[d];
74        sum -= freq[k2-d];
75        if (k1-d-1 >= 0) sum += freq[k1-d-1];
76    }
77    for (int d = 1; d <= mxd; d++) {
78        if (k1 <= d) validsum += distfreq[d];
79        freq[d] += distfreq[d], distfreq[d] = 0;
80    }
81    mxd = max(mxd, mxd);
82 }
83 for (int i = 1; i <= mxd; i++) freq[i] = 0;
84 }
85 };

```

5.2 Sum of distances

Given a tree, $f(u, v) :=$ distance from u to v in the tree, compute

$$\sum_{u,v} f(u, v)$$

. Time: $\mathcal{O}(n)$

```

1 vvi adj;
2 vi sum_going_down, sum_going_up, sz;
3
4 void dfs(int u, int p){
5     for (auto v : adj[u]){
6         if (v == p) continue;
7         dfs(v, u);
8         sz[u] += sz[v];
9         sum_going_down[u] += sum_going_down[v];
10    }
11    sum_going_down[u] += sz[u];
12 }
13
14 void dfs2(int u, int p, int par_ans){
15     int up_amount = sz[0] - sz[u];
16     sum_going_up[u] += par_ans + up_amount;
17     int sum = sum_going_down[u];
18     for (auto v : adj[u]){
19         if (v == p) continue;
20         int par_amount = sz[0] - sz[v];
21         // new p_ans is going up for is going up for p
22         // + go up to u and go down to other subtrees
23         dfs2(v, u, par_ans + par_amount + sum - (
            sum_going_down[v] + sz[v]));
24    }
25 }
26
27 void solve(){
28     adj = vvi(n);
29     sum_going_down = sum_going_up = vi(n);
30     sz = vi(n, 1);
31     // read adj
32     dfs(0, 0);
33     dfs2(0, 0, 0);
34
35     for (int i = 0; i < n; i++){
36         cout << sum_going_down[i] + sum_going_up[i] << ' ';
37     }
38     cout << endl;

```

39 }

5.3 Edge HLD

Sometimes the value is on the edges, for this few things need to change, but here is a template. Pre-computation: $\mathcal{O}(n)$ Queries: $\mathcal{O}(\log^2 n)$

```

1 struct EdgeHLD {
2     int n, timer = 0;
3     vvi adj;
4     vi parent, depth, size, heavy, head, value;
5     vi tin, tout;
6
7     segtree seg;
8
9     void init(int _n, vvi& _adj) {
10         n = _n;
11         adj = _adj;
12         value.assign(n, 0);
13         parent.assign(n, -1);
14         depth.assign(n, 0);
15         size.assign(n, 0);
16         heavy.assign(n, -1);
17         head.assign(n, 0);
18         tin.assign(n, 0);
19         tout.assign(n, 0);
20         timer = 0;
21
22         // edgeWeight[v] = weight of edge (parent[v], v),
23         // for v>0
24         // root (0) has no parent, so its value is dummy
25         // (0)
26         dfs1(0, 0, 0);
27         dfs2(0, 0);
28
29         vi linear(n);
30         for (int u = 0; u < n; u++)
31             linear[tin[u]] = value[u]; // position stores
32             // edge weight
33
34         seg.init(linear);
35     }
36
37     int dfs1(int u, int p, int w) {
38         size[u] = 1;
39         parent[u] = p;
40         value[u] = w;
41         int max_sz = 0;
42         for (auto [v, w] : adj[u]) {
43             if (v == p) continue;
44             depth[v] = depth[u] + 1;
45             int sz = dfs1(v, u, w);
46             size[u] += sz;
47             if (sz > max_sz) {
48                 max_sz = sz;
49                 heavy[u] = v;
50             }
51         }
52         return size[u];
53     }
54
55     void dfs2(int u, int h) {
56         tin[u] = timer++;
57         head[u] = h;

```

```

57         if (heavy[u] != -1)
58             dfs2(heavy[u], h);
59         for (auto [v, w] : adj[u]) {
60             if (v != parent[u] && v != heavy[u])
61                 dfs2(v, v);
62         }
63         tout[u] = timer;
64     }
65
66     // u deve ser o filho
67     void update_edge(int u, int val) {
68         seg.set(tin[u], val);
69     }
70
71     void rangeUpdate(int u, int v, int x) {
72         while (head[u] != head[v]) {
73             if (depth[head[u]] < depth[head[v]]) swap(u, v);
74             seg.rangeUpdate(tin[head[u]], tin[u] + 1, x);
75             u = parent[head[u]];
76         }
77         if (depth[u] > depth[v]) swap(u, v);
78         seg.rangeUpdate(tin[u] + 1, tin[v] + 1, x); // +1
79         // to skip LCA's edge
80     }
81
82     void update_subtree(int u, int x) {
83         // updates all edges in subtree of u (skip incoming
84         // edge to u)
85         seg.rangeUpdate(tin[u] + 1, tout[u], x);
86     }
87
88     segtree::node query(int u, int v) {
89         segtree::node res = seg.NEUTRAL;
90         while (head[u] != head[v]) {
91             if (depth[head[u]] < depth[head[v]]) swap(u, v);
92             res = seg.merge(res, seg.query(tin[head[u]], tin[u] + 1));
93             u = parent[head[u]];
94         }
95         if (depth[u] > depth[v]) swap(u, v);
96         res = seg.merge(res, seg.query(tin[u] + 1, tin[v] + 1)); // skip LCA's edge
97         return res;
98     }
99
100     segtree::node query_subtree(int u) {
101         // query all edges in subtree of u
102         return seg.query(tin[u] + 1, tout[u]);
103     }

```

5.4 HLD - Heavy light decomposition

If you need to compute a function on a path in a tree and need to support value updates on nodes, HLD is the way. Pre-computation: $\mathcal{O}(n)$ Queries: $\mathcal{O}(\log^2 n)$

OBS: this implementation uses the same segtree as this notebook, with 0-indexing and open-closed interval convention. Ideally, just change the segtree to change the computed function, the HLD struct remains the same. OBS2: this template also supports mass updates (path/subtree) and subtree queries.

```

1 struct HLD {
2     int n, timer = 0;
3     vvi adj;
4     vi parent, depth, size, heavy, head, value;
5     vi tin, tout;
6
7     segtree seg;
8
9     void init(int _n, vvi& _value, vvi& _adj) {
10         n = _n;
11         adj = _adj;
12         value = _value;
13         parent.assign(n, -1);
14         depth.assign(n, 0);
15         size.assign(n, 0);
16         heavy.assign(n, -1);
17         head.assign(n, 0);
18         tin.assign(n, 0);
19         tout.assign(n, 0);
20         timer = 0;
21
22         dfs1(0);
23         dfs2(0, 0);
24
25         vi linear(n);
26         for (int u = 0; u < n; u++)
27             linear[tin[u]] = value[u];
28
29         seg.init(linear);
30     }
31
32     int dfs1(int u) {
33         size[u] = 1;
34         int max_sz = 0;
35         for (int v : adj[u]) {
36             if (v == parent[u]) continue;
37             parent[v] = u;
38             depth[v] = depth[u] + 1;
39             int sz = dfs1(v);
40             size[u] += sz;
41             if (sz > max_sz) {
42                 max_sz = sz;
43                 heavy[u] = v;
44             }
45         }
46         return size[u];
47     }
48
49     void dfs2(int u, int h) {
50         tin[u] = timer++;
51         head[u] = h;
52         if (heavy[u] != -1)
53             dfs2(heavy[u], h);
54         for (int v : adj[u]) {
55             if (v != parent[u] && v != heavy[u])
56                 dfs2(v, v);
57         }
58         tout[u] = timer;
59     }
60
61     void update(int u, int val) {
62         seg.set(tin[u], val);
63     }
64
65     void rangeUpdate(int u, int v, int x) {
66         while (head[u] != head[v]) {
67             if (depth[head[u]] < depth[head[v]]) swap(u, v);
68             seg.rangeUpdate(tin[head[u]], tin[u] + 1, x);
69             u = parent[head[u]];
70         }

```

```

71     if (depth[u] > depth[v]) swap(u, v);
72     seg.rangeUpdate(tin[u], tin[v] + 1, x);
73 }
74
75 void update_subtree(int u, int x) {
76     seg.rangeUpdate(tin[u], tout[u], x);
77 }
78
79 segtree::node query(int u, int v) {
80     segtree::node res = seg.NEUTRAL;
81     while (head[u] != head[v]) {
82         if (depth[head[u]] < depth[head[v]])
83             swap(u, v);
84         res = seg.merge(res, seg.query(tin[head[u]], tin[
85             u]+1));
86         u = parent[head[u]];
87     }
88     if (depth[u] > depth[v]) swap(u, v);
89     res = seg.merge(res, seg.query(tin[u], tin[v]+1));
90     return res;
91 }
92
93 segtree::node query_subtree(int u){
94     return seg.query(tin[u], tout[u]);
95 }

```

5.5 LCA - RMQ

Pre-computation: $\mathcal{O}(n \log n)$

Queries: $\mathcal{O}(1)$

```

1 struct LCARMQ {
2     SparseTable st;
3     vi tin, dep, et_nodes, et_deps;
4
5     LCARMQ(vvi &adj){
6         int n = adj.size(), timer = 0;
7         tin = dep = vi(n);
8         et_nodes.reserve(2*n);
9         et_deps.reserve(2*n);
10
11         auto dfs = [&](auto &&dfs, int u, int p) -> void {
12             et_nodes.push_back(u);
13             et_deps.push_back(dep[u]);
14             tin[u] = timer++;
15             for (int v : adj[u]) if (v != p) {
16                 dep[v] = dep[u]+1;
17                 dfs(dfs, v, u);
18                 et_nodes.push_back(u);
19                 et_deps.push_back(dep[u]);
20             }
21             timer++;
22         };
23         dfs(dfs, 0, 0);
24
25         st.build(et_deps);
26     }
27
28     int get(int u, int v){
29         int tu = tin[u], tv = tin[v];
30         if (tu > tv) swap(tu, tv);
31         auto [val, id] = st.min(tu, tv);
32         return et_nodes[id];
33     }
34 };

```

5.6 LCA - binary lifting

Pre-computation: $\mathcal{O}(n \log n)$ Queries: $\mathcal{O}(\log n)$ OBS: just call `dfs(root)` before starting queries.

```

1 vvi adj, up;
2 vi tin, tout;
3 int timer = 0;
4
5 void dfs(int u, int p){
6     tin[u] = timer++;
7     for (auto v : adj[u]){
8         if (v == p) continue;
9         up[v][0] = u;
10        for (int dist = 1; dist < LOGN; dist++){
11            up[v][dist] = up[up[v][dist-1]][dist-1];
12        }
13        dfs(v);
14    }
15    tout[u] = timer++;
16 }
17
18 int isAncestor(int u, int v){
19     return tin[u] <= tin[v] && tout[v] <= tout[u];
20 }
21
22 int lca(int u, int v){
23     if (isAncestor(u, v)) return u;
24     if (isAncestor(v, u)) return v;
25     for (int dist = LOGN-1; dist >= 0; dist--){
26         if (!isAncestor(up[u][dist], v)) u = up[u][dist];
27     }
28     return up[u][0];
29 }

```

6 Problemas clássicos

6.1 2SAT

Struct for solving 2SAT problems that supports many types of boolean expressions. To add a negated literal use `u`

```

1 // para adicionar negacao usar ~u
2 // Ex: a clausula (a v !b) se traduz para add_or(a, ~b)
3 struct TwoSatSolver {
4     int n;
5     vvi adj, adjT;
6     vector<bool> vis, assignment;
7     vi topo, scc;
8
9     void build(int _n){
10         n = 2*_n;
11         adj.assign(n, vi());
12         adjT.assign(n, vi());
13     }
14
15     int get(int u){
16         if (u < 0) return 2*(~u)+1;
17         else return 2*u;
18     }
19
20     // u -> v
21     void add_impl(int u, int v){
22         u = get(u), v = get(v);
23         adj[u].push_back(v);

```

```

24         adjT[v].push_back(u);
25         adj[v^1].push_back(u^1);
26         adjT[u^1].push_back(v^1);
27     }
28
29     // u || v
30     void add_or(int u, int v){
31         add_impl(~u, v);
32     }
33
34     // u && v
35     void add_and(int u, int v){
36         add_or(u, u); add_or(v, v);
37     }
38
39     // u ^ v (equiv of x != v)
40     void add_xor(int u, int v){
41         add_impl(u, ~v);
42         add_impl(~u, v);
43     }
44
45     // u == v
46     void add_equals(int u, int v){
47         add_impl(u, v);
48         add_impl(v, u);
49     }
50
51     void toposort(int u){
52         vis[u] = true;
53         for (int v : adj[u])
54             if (!vis[v]) toposort(v);
55         topo.push_back(u);
56     }
57
58     void dfs(int u, int c){
59         scc[u] = c;
60         for (int v : adjT[u])
61             if (!scc[v]) dfs(v, c);
62     }
63
64     pair<bool, vector<bool>> solve(){
65         topo.clear();
66         vis.assign(n, false);
67
68         for (int i = 0; i < n; i++)
69             if (!vis[i]) toposort(i);
70
71         reverse(topo.begin(), topo.end());
72
73         scc.assign(n, 0);
74         int c = 0;
75         for (int u : topo)
76             if (!scc[u]) dfs(u, ++c);
77
78         assignment.assign(n/2, false);
79         for (int i = 0; i < n; i += 2){
80             if (scc[i] == scc[i+1]) return {false, {}};
81             assignment[i/2] = scc[i] > scc[i+1];
82         }
83
84         return {true, assignment};
85     }
86 };

```

6.2 Next Greater Element

One of the classic stack applications. Easy to translate to

lower, leq or geq, just change the comparator of the while.

```
1 vi ngt(n, n);
2 stack<ii> st;
3 for (int i = 0; i < n; i++){
4     while (!st.empty() && st.top().first < h[i]){
5         ngt[st.top().second] = i;
6         st.pop();
7     }
8     st.emplace(h[i], i);
9 }
```

7 Strings

7.1 Hashing

Creation time: $\mathcal{O}(n)$ Access time: $\mathcal{O}(1)$ Space: $\mathcal{O}(n)$

```
1 namespace DoubleHashing {
2     const int MOD1 = 2147483647, MOD2 = 2147483629;
3     using mint1 = mint<MOD1>;
4     using mint2 = mint<MOD2>;
5     vector<mint1> pmod1{1}, invpmod1{1};
6     vector<mint2> pmod2{1}, invpmod2{1};
7
8     const int MAX_VAL = 256;
9     const int OFFSET = 'a' - 1;
10    mt19937 rng(chrono::steady_clock::now().
11        time_since_epoch().count());
12    const mint1 p1 = uniform_int_distribution<int>(
13        MAX_VAL+1, MOD1-1)(rng);
14    const mint2 p2 = uniform_int_distribution<int>(
15        MAX_VAL+1, MOD2-1)(rng);
16    const mint1 inv1 = mint1::inv(p1);
17    const mint2 inv2 = mint2::inv(p2);
18
19    void extend(int n){
20        while(pmod1.size() <= n){
21            pmod1.push_back(pmod1.back()*p1);
22            pmod2.push_back(pmod2.back()*p2);
23            invpmod1.push_back(invpmod1.back()*inv1);
24            invpmod2.push_back(invpmod2.back()*inv2);
25        }
26    }
27
28    struct Hash {
29        mint1 h1;
30        mint2 h2;
31        int len = 0;
32        Hash apply(int x) const {
33            extend(len+1);
34            return {
35                h1+pmod1[len]*(x-OFFSET),
36                h2+pmod2[len]*(x-OFFSET),
37                len+1
38            };
39        }
40
41        bool operator<(const Hash &o) const {
42            if (h1.val != o.h1.val) return h1.val < o.h1.val;
43            if (h2.val != o.h2.val) return h2.val < o.h2.val;
44            return len < o.len;
45        }
46
47        bool operator==(const Hash &o) const {
48            return h1.val == o.h1.val && h2.val == o.h2.val
49                && len == o.len;
50        }
51    };
52 }
```

```
46 struct PrefixHash {
47     vector<Hash> pref;
48     PrefixHash(const string &s){
49         int n = s.length();
50         pref.reserve(n+1);
51         pref.emplace_back();
52         for (auto c : s)
53             pref.push_back(pref.back().apply(c));
54     }
55     Hash get(int l, int r) const {
56         l++, r++;
57         mint1 h1 = (pref[r].h1 - pref[l-1].h1)*invpmod1[
58             -1];
59         mint2 h2 = (pref[r].h2 - pref[l-1].h2)*invpmod2[
60             -1];
61         return {h1, h2, r-l+1};
62     }
63 };
64 }
```

7.2 KMP

```
1 vi compute_lps(const string &pat){
2     int m = pat.length();
3     vi lps(m);
4     int len = 0;
5     for (int i = 1; i < m; i++){
6         while(len > 0 && pat[i] != pat[len])
7             len = lps[len-1];
8         if (pat[i] == pat[len]) len++;
9         lps[i] = len;
10    }
11    return lps;
12 }
13
14 // find all occurrences
15 vi kmp_search(const string &txt, const string &pat){
16     int n = txt.length();
17     int m = pat.length();
18     if (m == 0) return {};
19     vi lps = compute_lps(pat);
20     vi occurrences;
21     int j = 0;
22     for (int i = 0; i < n; i++){
23         while (j > 0 && txt[i] != pat[j])
24             j = lps[j-1];
25         if (txt[i] == pat[j]) j++;
26
27         if (j == m) {
28             occurrences.push_back(i-m+1);
29             j = lps[j-1];
30         }
31     }
32     return occurrences;
33 }
34
35 // find all occurrences (simpler version)
36 vi kmp_search(const string &txt, const string &pat){
37     int n = txt.length(), m = pat.length();
38     vi lps = compute_lps(pat + '#' + txt);
39     vi occurrences;
40     for (int i = 0; i < n+m+1; i++){
41         if (lps[i] == pat.length())
42             occurrences.push_back(i-m*2);
43     }
44     return occurrences;
45 }
46 }
```

```
47 // borda sao os prefixos que tambem sao sufixos
48 vi find_borders(const string &s){
49     vi lps = compute_lps(s);
50     int i = s.length()-1;
51
52     vi ans;
53     while (lps[i] > 0){
54         ans.push_back(lps[i]);
55         i = lps[i]-1;
56     }
57     reverse(ans.begin(), ans.end());
58     return ans;
59 }
```

7.3 Suffix Array

Time: $\mathcal{O}(n \log n)$ Space: $\mathcal{O}(n)$

```
1 struct SuffixArray {
2     vi sa, lcp;
3     SuffixArray(vi s, int lim = 256) {
4         s.push_back(0);
5         int n = s.size(), k = 0, a, b;
6         vi x(all(s), y(n), ws(max(n, lim)));
7         sa = lcp = y, iota(all(sa), 0);
8         for (int j = 0, p = 0; p < n; j = max(1ll, 2 * j),
9             lim = p) {
10             p = j, iota(all(y), n - j);
11             for (int i = 0; i < n; i++)
12                 if (sa[i] >= j)
13                     y[p++] = sa[i] - j;
14             fill(all(ws), 0);
15             for (int i = 0; i < n; i++) ws[x[i]]++;
16             for (int i = 1; i < lim; i++) ws[i] += ws[i-1];
17             for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
18             swap(x, y), p = 1, x[sa[0]] = 0;
19             for (int i = 1; i < n; i++) {
20                 a = sa[i-1], b = sa[i];
21                 x[b] = (y[a] == y[b] && y[a+j] == y[b+j]) ?
22                     p-1 : p++;
23             }
24             for (int i = 0, j; i < n-1; lcp[x[i++]] = k)
25                 for (k && k--, j = sa[x[i]-1]; s[i+k] == s[j+k]; k++);
26         }
27     }
28 }
```

7.4 Suffix Automaton

```
1 struct SAM {
2     struct State {
3         int len, link;
4         ll cnt = 0;
5         int first_occ=-1;
6         vi next;
7     };
8
9     static const int alphabet = 26;
10    vector<State> st;
11    int last;
12
13    SAM(string s){
14        st.push_back({0,-1,0,-1,vi(alphabet,-1)});
15        last = 0;
16        for (int i = 0; i < s.length(); i++){
17            extend(s[i], i);
18        }
19    }
```

```

18     }
19     calc_cnt();
20 }
21
22 void extend(char c, int id){
23     int cur = st.size();
24     st.push_back({st[last].len+1,0,1,id,vi(alphabet,-1)
25     });
26     int p = last;
27     while(p!=-1 && st[p].next[c-'a']==-1){
28         st[p].next[c-'a'] = cur;
29         p = st[p].link;
30     }
31     if (p == -1){
32         st[cur].link = 0;
33         last = cur;
34         return;
35     }
36     int q = st[p].next[c-'a'];
37     if (st[p].len+1 == st[q].len) {
38         st[cur].link = q;
39         last = cur;
40         return;
41     }
42     int clone = st.size();
43     st.push_back({
44         st[p].len+1,
45         st[q].link,
46         0,
47         st[q].first_occ,
48         st[q].next
49     });
50     while (p!=-1 && st[p].next[c-'a'] == q){
51         st[p].next[c-'a'] = clone;
52         p = st[p].link;
53     }
54     st[q].link = st[cur].link = clone;
55     last = cur;
56 }
57
58 void calc_cnt() {
59     int sz = st.size();
60     int mx = 0;
61     for (auto &s : st) mx = max(mx, s.len);
62
63     vi c(mx+1);
64     vi nodes(sz);
65
66     for (int i = 0; i < sz; i++) c[st[i].len]++;
67     for (int i = 1; i <= mx; i++) c[i] += c[i-1];
68     for (int i = 0; i < sz; i++) nodes[--c[st[i].len]]
69         = i;
70
71     for (int i = sz-1; i >= 0; i--) {
72         int u = nodes[i];
73         if (st[u].link != -1) st[st[u].link].cnt += st[
74             u].cnt;
75     }
76
77     ll count_occurrences(string t){
78         ll cur = 0;
79         for (char c : t){
80             if (st[cur].next[c-'a'] == -1) return 0;
81             cur = st[cur].next[c-'a'];
82         }
83         return st[cur].cnt;
84     }

```

```

85     int first_occurrence(string t){
86         int cur = 0;
87         for (char c : t){
88             if (st[cur].next[c-'a'] == -1) return -2;
89             cur = st[cur].next[c-'a'];
90         }
91         return st[cur].first_occ-t.length()+1;
92     }
93
94     ll distinct_substrings(){
95         ll ans = 0;
96         for (int i = 1; i < st.size(); i++){
97             ans += st[i].len - st[st[i].link].len;
98             return ans;
99         }
100
101         vector<ll> distinct_substrings_perlen(int n){
102             vector<ll> diff(n+2);
103             for (int i = 1; i < st.size(); i++){
104                 int l = st[st[i].link].len+1;
105                 int r = st[i].len;
106                 diff[l]++; diff[r+1]--;
107             }
108             vector<ll> ans(n+1);
109             ans[0] = diff[0];
110             for (int i = 1; i <= n; i++){
111                 ans[i] = ans[i-1]+diff[i];
112             }
113             return ans;
114         }
115     }
116
117     vector<ll> dp;
118     void calc_paths(int u){
119         if (dp[u] != -1) return;
120         dp[u]=1;
121         for (int c = 0; c < alphabet; c++){
122             int v = st[u].next[c];
123             if (v == -1) continue;
124             calc_paths(v);
125             dp[u] += dp[v];
126         }
127     }
128
129     string find_kth(int k){
130         dp.assign(st.size(),-1);
131         calc_paths(0);
132         int u = 0;
133         string ans = "";
134         while(k>0){
135             for (int c = 0; c < alphabet; c++){
136                 int v = st[u].next[c];
137                 if (v== -1) continue;
138                 bool ok = false;
139                 if (k <= dp[v]){
140                     ans += 'a'+c;
141                     u = v;
142                     k--;
143                     ok = true;
144                     break;
145                 }
146                 if (!ok) k-=dp[v];
147             }
148         }
149         return ans;
150     }
151
152     void calc_paths_with_repetitions(int u){
153         if (dp[u] != -1) return;
154         dp[u]=st[u].cnt;

```

```

155         for (int c = 0; c < alphabet; c++){
156             int v = st[u].next[c];
157             if (v== -1) continue;
158             calc_paths_with_repetitions(v);
159             dp[u] += dp[v];
160         }
161     }
162
163     string find_kth_with_repetitions(ll k){
164         dp.assign(st.size(),-1);
165         calc_paths_with_repetitions(0);
166         int u = 0;
167         string ans = "";
168         while(k>0){
169             for (int c = 0; c < alphabet; c++){
170                 int v = st[u].next[c];
171                 if (v== -1) continue;
172                 bool ok = false;
173                 if (k <= dp[v]){
174                     ans += 'a'+c;
175                     k-=st[v].cnt;
176                     u = v;
177                     ok = true;
178                     break;
179                 }
180                 if (!ok) k-=dp[v];
181             }
182         }
183         return ans;
184     }
185 };

```

7.5 Z

$$z[i] := \max(k) | s[0..k-1] = s[i..i+k-1]$$

Time: $\mathcal{O}(n+m)$ Space: $\mathcal{O}(n+m)$

```

1 vi compute_z(const string &s) {
2     int n = s.length();
3     vi z(n);
4     int l = 0, r = 0;
5
6     for (int i = 1; i < n; i++) {
7         if (i <= r)
8             z[i] = min(r - i + 1, z[i-l]);
9
10        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
11            z[i]++;
12        if (i + z[i] - 1 > r) {
13            l = i;
14            r = i + z[i] - 1;
15        }
16    }
17    return z;
18 }
19
20 vi find_occurrences(const string &txt, const string &
21     pat){
22     vi occurrences;
23     vi z = compute_z(pat + '#' + txt);
24     int n = txt.length(), m = pat.length();
25     for (int i = 0; i < n+m+1; i++){
26         if (z[i] == m) occurrences.push_back(i-m-1);
27     }
28     return occurrences;
29 }

```

8 Math

8.1 Combinatorics (Pascal's Triangle)

Computes "n choose k". Requires factorials to be pre-computed. Time: $\mathcal{O}(\log ZAP)$

8.1.1 Combinatorial Analysis

Fundamental Counting Principles

- **Permutations:** The number of ways to arrange k items from a set of n distinct items.

$$P(n, k) = \frac{n!}{(n - k)!}$$

- **Combinations (Binomial Coefficient):** The number of ways to choose k items from a set of n distinct items, regardless of order.

$$\binom{n}{k} = C(n, k) = \frac{n!}{k!(n - k)!}$$

- **Combinations with Repetition (Stars and Bars):** The number of ways to choose k items of n types, allowing repetitions. Equivalently, the number of ways to distribute k identical balls into n distinct urns.

$$\binom{k + n - 1}{n - 1} = \binom{k + n - 1}{k}$$

Binomial Coefficient Properties and Pascal's Triangle

- **Pascal's Triangle**

$$\begin{array}{c} \binom{0}{0} \\ \binom{1}{0} \binom{1}{1} \\ \binom{2}{0} \binom{2}{1} \binom{2}{2} \\ \binom{3}{0} \binom{3}{1} \binom{3}{2} \binom{3}{3} \\ \binom{4}{0} \binom{4}{1} \binom{4}{2} \binom{4}{3} \binom{4}{4} \end{array}$$

- **Stifel's Relation:** Each element in Pascal's Triangle is the sum of the two elements immediately above it.

$$\binom{n}{k} = \binom{n - 1}{k} + \binom{n - 1}{k - 1}$$

- **Symmetry:** Elements of a row are symmetric with respect to the center. Choosing k elements is the same as choosing the $n - k$ elements to be left behind.

$$\binom{n}{k} = \binom{n}{n - k}$$

- **Row Sum:** The sum of all elements in row n of Pascal's Triangle (where the first row is $n = 0$) is equal to 2^n .

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

- **Hockey Stick Identity:** The sum of elements in a diagonal, starting at

$$\binom{r}{r}$$

and ending at

$$\binom{n}{r}$$

, is equal to the element in the next row and next column,

$$\binom{n + 1}{r + 1}$$

$$\sum_{i=r}^n \binom{i}{r} = \binom{n + 1}{r + 1}$$

- **Binomial Theorem:**

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$$

- **Vandermonde's Identity:**

$$\sum_{j=0}^k \binom{m}{j} \binom{n}{k-j} = \binom{m+n}{k}$$

The easiest way to understand the identity is through a counting problem. Imagine you have a committee with m men and n women. How many ways can you form a subcommittee of k people?

Way 1 Direct Counting

You have a total of $m+n$ people and need to choose k of them. The number of ways to do this is simply:

$$\binom{m+n}{k}$$

Way 2 Counting by Cases

We can divide the problem into cases, based on how many men (j) are chosen for the subcommittee.

Case 0: Choose 0 men and k women. The number of ways is

$$\binom{m}{0} \binom{n}{k}$$

.

Case 1: Choose 1 man and $k-1$ women. The number of ways is

$$\binom{m}{1} \binom{n}{k-1}$$

.

Case j : Choose j men and $k-j$ women. The number of ways is

$$\binom{m}{j} \binom{n}{k-j}$$

.

Other Important Concepts

- **Catalan Numbers:** A sequence of natural numbers that occurs in various counting problems (e.g., number of binary trees, balanced parenthesis expressions).

$$C_n = \binom{2n}{n} - \binom{2n}{n+1} = \frac{1}{n+1} \binom{2n}{n}$$

A commonly used combinatorial proof for the Catalan numbers involves counting the number of lattice (grid) paths from $(0,0)$ to (n,n) that do not cross above the diagonal $y = x$. Each such path consists of n rightward steps and n upward steps, and the Catalan number counts the number of these "Dyck paths" that never go above the diagonal.

- **Stirling Numbers of the Second Kind:** The number of ways to partition a set of n labeled objects into k non-empty unlabeled subsets. Denoted by $S(n, k)$ or

$$\{n \atop k\}$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

The Stirling numbers of the second kind can also be computed recursively:

$$S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1)$$

with the boundary conditions:

$$S(0, 0) = 1; \quad S(n, 0) = 0 \text{ for } n > 0; \quad S(0, k) = 0 \text{ for } k > 0$$

- **Bell Number:** The Bell number B^n counts the total number of ways to partition a set of n labeled elements into any number (from 1 up to n) of non-empty, unlabeled subsets. It can also be written as a recurrence relation

$$B^n = \sum_{k=0}^n S(n, k)$$

- **Pigeonhole Principle:** If n items are put into m boxes, with $n > m$, then at least one box must contain more than one item.

8.2 Convolutions

8.2.1 AND convolution

$$c[k] = \sum_{i \& j = k} a[i] \cdot b[j]$$

```
1 vector<mint> and_conv(vector<mint> a, vector<mint> b){
2   int n = a.size(); // must be pow of 2
3   for (int j = 1; j < n; j <= 1) {
4     for (int i = 0; i < n; i++) {
5       if (i&j) {
6         a[i~j] += a[i];
7         b[i~j] += b[i];
8       }
9     }
10  }
11  for (int i = 0; i < n; i++) a[i] *= b[i];
12  for (int j = 1; j < n; j <= 1)
13    for (int i = 0; i < n; i++)
14      if (i&j) a[i~j] -= a[i];
15  return a;
16 }
```

8.2.2 GCD convolution

$$c[k] = \sum_{\gcd(i,j)=k} a[i] \cdot b[j]$$

```
1 vector<mint> gcd_conv(vi a, vi b){
2   int n = (int)max(a.size(), b.size());
3   a.resize(n); b.resize(n);
4   vector<mint> c(n);
5   for (int i = 1; i < n; i++) {
6     mint x = 0;
7     mint y = 0;
8     for (int j = i; j < n; j += i) {
9       x += a[j];
10      y += b[j];
11    }
12    c[i] = x*y;
13  }
14  for (int i = n-1; i >= 1; i--)
15    for (int j = 2*i; j < n; j += i)
16      c[i] -= c[j];
17  return c;
18 }
```

8.2.3 LCM convolution

$$c[k] = \sum_{\text{lcm}(i,j)=k} a[i] \cdot b[j]$$

```
1 vector<mint> lcm_conv(vi a, vi b){
2   int n = (int)max(a.size(), b.size());
3   a.resize(n); b.resize(n);
4   vector<mint> c(n), x(n), y(n);
5   for (int i = 1; i < n; i++) {
6     for (int j = i; j < n; j += i) {
7       x[j] += a[i];
8       y[j] += b[i];
9     }
10  }
```

```
10   c[i] = x[i]*y[i];
11 }
12 for (int i = 1; i < n; i++)
13   for (int j = 2 * i; j < n; j += i)
14     c[j] -= c[i];
15 return c;
16 }
```

8.2.4 OR convolution

$$c[k] = \sum_{i|j=k} a[i] \cdot b[j]$$

```
1 vector<mint> or_conv(vector<mint> a, vector<mint> b){
2   int n = a.size(); // must be pow of 2
3   for (int j = 1; j < n; j <= 1) {
4     for (int i = 0; i < n; i++) {
5       if (i&j) {
6         a[i] += a[i~j];
7         b[i] += b[i~j];
8       }
9     }
10  }
11  for (int i = 0; i < n; i++) a[i] *= b[i];
12  for (int j = 1; j < n; j <= 1)
13    for (int i = 0; i < n; i++)
14      if (i&j) a[i] -= a[i~j];
15  return a;
16 }
```

8.2.5 XOR convolution

$$c[k] = \sum_{i \oplus j = k} a[i] \cdot b[j]$$

```
1 void fwht(vector<mint> &a, bool inv){
2   int n = a.size(); // must be pow of 2
3   for (int step = 1; step < n; step <= 1){
4     for (int i = 0; i < n; i += 2*step) {
5       for (int j = i; j < i+step; j++){
6         auto u = a[j];
7         auto v = a[j+step];
8         a[j] = u+v;
9         a[j+step] = u-v;
10      }
11    }
12  }
13  if (inv) for (auto &x : a) x /= n;
14 }
15 vector<mint> xor_conv(vector<mint> a, vector<mint> b){
16   int n = a.size();
17   fwht(a,0), fwht(b, 0);
18   for (int i = 0; i < n; i++) a[i] *= b[i];
19   fwht(a,1);
20   return a;
21 }
```

8.3 Extended Euclid

Time: $\mathcal{O}(\log n)$.

```
1 int extended_gcd(int a, int b, int &x, int &y) {
2     x = 1, y = 0;
3     int x1 = 0, y1 = 1;
4     while (b) {
5         int q = a / b;
6         tie(x, x1) = make_tuple(x1, x - q * x1);
7         tie(y, y1) = make_tuple(y1, y - q * y1);
8         tie(a, b) = make_tuple(b, a - q * b);
9     }
10    return a;
11 }
```

8.4 Factorization

Time: $\mathcal{O}(\sqrt{n})$

```
1 // O(sqrt(n)) fatores repetidos
2 vi fatora(int n) {
3     vi factors;
4     for (int x = 2; x*x <= n; x++) {
5         while (n%x == 0) {
6             factors.push_back(x);
7             n /= x;
8         }
9     }
10    if (n > 1) factors.push_back(n);
11    return factors;
12 }
```

8.5 FFT - Fast Fourier Transform

Divide and conquer algorithm used for convolutions and polynomial multiplication. Vector size a is a power of 2.
Time: $\mathcal{O}(n \log n)$ Space: $\mathcal{O}(n)$

```
1 void fft(vector<cd> &a, bool invert){
2     int len = a.size();
3     for(int i = 1, j = 0; i < len; i++){
4         int bit = len >> 1;
5         while(bit & j){
6             j ^= bit;
7             bit >>= 1;
8         }
9         j ^= bit;
10        if(i < j) swap(a[i], a[j]);
11    }
12    for(int l = 2; l <= len; l <= 1){
13        double ang = 2*PI/l * (invert ? -1 : 1);
14        cd wd(cos(ang), sin(ang));
15        for(int i = 0; i < len; i += l){
16            cd w(1);
17            for(int j = 0; j < l/2; j++){
18                cd u = a[i+j], v = a[i+j+l/2];
19                a[i+j] = u+w*v;
20                a[i+j+l/2] = u-w*v;
21                w *= wd;
22            }
23        }
24    }
25    if(invert){
```

```
26        for(int i = 0; i < len; i++){
27            a[i] /= len;
28        }
29    }
30 }
```

8.6 Inclusion-Exclusion Principle

TODO: rewrite math statement

```
1 // Exemplo:
2 // Contar numeros de 1 a n divisiveis por uma lista de
3 // primos.
4 int n;
5 vi primes;
6 int factors = primes.size();
7 int total_divisible = 0;
8 // Itera pelas bitmasks nao vazias de 'primes'
9 for (int i = 1; i < (1 << factors); i++) {
10    int current_lcm = 1;
11    int subset_size = 0;
12
13    // calcula lcm do subconjunto
14    for (int j = 0; j < factors; j++) {
15        if (i & (1<<j)) {
16            subset_size++;
17            current_lcm = lcm(current_lcm, primes[j]);
18            if (current_lcm > n) break;
19        }
20    }
21
22    if (current_lcm > n) {
23        continue;
24    }
25
26    int count = n / current_lcm;
27
28    // Aplica o Principio da Inclusao-Exclusao:
29    // Se o tamanho do subconjunto eh impar, adiciona.
30    // Se o tamanho do subconjunto eh par, subtrai.
31    if (subset_size & 1) {
32        total_divisible += count;
33    } else {
34        total_divisible -= count;
35    }
36 }
```

8.7 Legendre's formula

Computes the largest power of a prime p in $n!$

```
1 int legendre(int n, int p){
2     int ans = 0;
3     while(n>0){
4         n /= p; ans += n;
5     }
6     return ans;
7 }
```

8.8 Linear Sieve + Möbius Function

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1 \\ (-1)^k & \text{if } n \text{ is a product of } k \text{ distinct primes} \\ 0 & \text{if } p^2 \mid n \text{ for some prime } p \end{cases}$$

8.8.1 Applications

- **Inclusion-Exclusion:** Naturally generates $1, -1, 0$ coefficients for prime factor sets.
- **Möbius Inversion:** Converts $g(n) = \sum_{d|n} f(d)$ to $f(n) = \sum_{d|n} \mu(d)g(\frac{n}{d})$. Often reduces $\mathcal{O}(N)$ divisor sums to $\mathcal{O}(\sqrt{N})$.
- **Dirichlet Convolution:** $\mu * 1 = \epsilon$. Core to sub-linear prefix sum algorithms (e.g., DuSieve) yielding $\mathcal{O}(N^{2/3})$ time.

```
1 auto sieve = [](int n) -> pair<vi, vi> {
2     vi primes, mu(n+1);
3     mu[1] = 1;
4     vector<bool> is_prime(n+1, true);
5     is_prime[0] = is_prime[1] = false;
6     for (int i = 2; i <= n; ++i) {
7         if (is_prime[i]) {
8             primes.push_back(i);
9             mu[i] = -1;
10        }
11        for (int p : primes) {
12            if (i*p > n) break;
13            is_prime[i*p] = false;
14            if (i%p == 0) {
15                mu[i*p] = 0;
16                break;
17            }
18            mu[i*p] = -mu[i];
19        }
20    }
21    return {primes, mu};
22 };
```

8.9 Matrix template

A template for square matrices, used for solving linear recurrences with fast exponentiation, memory is on a 1D vector, but can be accessed with $A[i][j]$ normally (custom operator[])

```
1 template<typename T>
2 struct mat{
3     vector<T> m;
4     int n;
5
6     mat(int _n = 0, bool identity = false) : n(_n) {
7         m.resize(n*n);
8         if (!identity) return;
9         for (int i = 0; i < n; i++)
10            m[i*n+i] = 1;
11     }
```

```

11 }
12
13 mat& operator+=(const mat& o){
14     for (int i = 0; i < n; i++){
15         int ra = i*n;
16         for (int j = 0; j < n; j++){
17             m[ra+j] += o.m[ra+j];
18         }
19     }
20     return *this;
21 }
22 mat& operator-=(const mat& o){
23     for (int i = 0; i < n; i++){
24         int ra = i*n;
25         for (int j = 0; j < n; j++){
26             m[ra+j] -= o.m[ra+j];
27         }
28     }
29     return *this;
30 }
31 mat& operator*=(const mat& o){
32     vector<T> ans(n*n);
33     for (int i = 0; i < n; i++){
34         for (int k = 0; k < n; k++){
35             int ra = i*n, rb = k*n;
36             for (int j = 0; j < n; j++){
37                 ans[ra+j] += m[ra+k] * o.m[rb+j];
38             }
39         }
40     }
41     this->m = ans;
42     return *this;
43 }
44 friend mat operator+(mat a, const mat& b){
45     return a+b;
46 }
47 friend mat operator-(mat a, const mat& b){
48     return a-b;
49 }
50 friend mat operator*(mat a, const mat& b){
51     return a*b;
52 }
53 T* operator[](int i){
54     return &m[i*n];
55 }
56 vector<T> operator*(const vector<T> &v){
57     vector<T> ans(n);
58     for (int i = 0; i < n; i++){
59         int ra = i*n;
60         for (int j = 0; j < n; j++){
61             ans[i] += m[ra+j]*v[j];
62         }
63     }
64     return ans;
65 }
66 mat operator^(int e){
67     return exp(*this, e);
68 }
69 static mat exp(mat b, ll e){
70     mat ans = mat(b.n, true);
71     while(e>0){
72         if(e&1) ans*=b;
73         b*=b;
74         e>>=1;
75     }
76     return ans;
77 };

```

8.10 Mint

```

1 template<const int MOD>
2 struct mint {
3     int val;
4     mint(ll v=0){
5         ll x = v%MOD;
6         if (x<0) x+=MOD;
7         val = x;
8     }
9     mint& operator+=(const mint& o){
10        ll x = (ll)val + o.val;
11        if (x >= MOD) x -= MOD;
12        val = x;
13        return *this;
14    }
15    mint& operator-=(const mint& o){
16        ll x = (ll)val - o.val;
17        if (x < 0) x += MOD;
18        val = x;
19        return *this;
20    }
21    mint& operator*=(const mint& o){
22        val = (ll)val*o.val%MOD;
23        return *this;
24    }
25    mint& operator/=(const mint& o){
26        val = (ll)val*inv(o).val%MOD;
27        return *this;
28    }
29    friend mint operator+(mint a, const mint& b) { return
        a+b; }
30    friend mint operator-(mint a, const mint& b) { return
        a-b; }
31    friend mint operator*(mint a, const mint& b) { return
        a*b; }
32    friend mint operator/(mint a, const mint& b) { return
        a/b; }
33    static mint power(mint b, int e){
34        mint ans = 1;
35        while(e>0){
36            if (e&1) ans *= b;
37            b*=b; e>>=1;
38        }
39        return ans;
40    }
41    static mint inv(mint n){ return power(n,MOD-2); }
42 };

```

8.11 Modular Inverse

If m is prime, can use binary exponentiation to compute a^{p-2} (Fermat's Little Theorem).

This code works for non-prime m , as long as it is coprime to a .

Time: $\mathcal{O}(\log m)$

```

1 int modInverse(int a, int m) {
2     int x, y;
3     int g = extendedGcd(a, m, x, y);
4     if (g != 1) return -1;
5     return (x % m + m) % m;
6 }

```

8.12 Number Theoretic Transform (NTT)

NTT is a fast algorithm (analogous to FFT) for polynomial multiplication modulo a special prime. It requires a prime modulus $p = c \cdot 2^k + 1$ (a "primitive root prime") and a primitive 2^k -th root of unity modulo p .

- **Prime Choices:** To use NTT, pick a modulus and a matching primitive root (see table below). For arbitrary moduli (e.g., $10^9 + 7$), multiply with several NTT-friendly primes and reconstruct with CRT (see `crt_multiply`).
- **Time Complexity:** $\mathcal{O}(n \log n)$ for polynomial multiplication.

8.12.1 NTT-Friendly Primes and Roots

NTT-friendly primes and their primitive roots:

- Mod: 998244353, Root: 3, Max N: 2^{23}
- Mod: 734003201, Root: 3, Max N: 2^{20}
- Mod: 167772161, Root: 3, Max N: 2^{25}
- Mod: 469762049, Root: 3, Max N: 2^{26}

Use the modulus as MOD and the root as ROOT when instantiating the NTT.

- For large/concrete moduli, see `crt_multiply` in the code for a multi-modulus solution with Chinese Remainder Theorem (CRT).

```

1 namespace NTT{
2     template<typename T, ll MOD, ll ROOT>
3     void transform(vector<T>& a, bool invert) {
4         int n = a.size();
5
6         for (int i = 1, j = 0; i < n; i++) {
7             int bit = n >> 1;
8             for (; j & bit; bit >>= 1) j ^= bit;
9             j ^= bit;
10            if (i < j) swap(a[i], a[j]);
11        }
12
13        for (int len = 2; len <= n; len <= 1) {
14            T wlen = T::power(ROOT, (MOD - 1) / len);
15            if (invert) wlen = T::inv(wlen);
16            for (int i = 0; i < n; i += len) {
17                T w = 1;
18                for (int j = 0; j < len / 2; j++) {
19                    T u = a[i + j], v = a[i + j + len / 2] * w;
20                    a[i + j] = u + v;
21                    a[i + j + len / 2] = u - v;
22                    w *= wlen;
23                }
24            }
25        }
26
27        if (invert) {
28            T n_inv = T::inv(n);

```

```

29     for (T& x : a) x *= n_inv;
30 }
31 }
32
33 template<typename T, ll MOD, ll ROOT>
34 vector<ll> multiply(const vector<ll>& a, const vector
35 <ll>& b) {
36     vector<T> fa(a.begin(), a.end()), fb(b.begin(), b.
37 end());
38     int n = 1;
39     while (n < a.size() + b.size()) n <= 1;
40     fa.resize(n);
41     fb.resize(n);
42
43     transform<T, MOD, ROOT>(fa, false);
44     transform<T, MOD, ROOT>(fb, false);
45
46     for (int i = 0; i < n; i++) fa[i] *= fb[i];
47
48     transform<T, MOD, ROOT>(fa, true);
49
50     vector<ll> result(n);
51     for (int i = 0; i < n; i++) result[i] = fa[i].val;
52     return result;
53 }
54
55 vector<ll> crt_multiply(const vector<ll>& a, const
56 vector<ll>& b) {
57     const ll mod1 = 998244353;
58     const ll root1 = 3;
59     using mint1 = mint<mod1>;
60     vector<ll> ans1 = NTT::multiply<mint1, mod1, root1
61 >(a, b);
62
63     const ll mod2 = 1004535809;
64     const ll root2 = 3;
65     using mint2 = mint<mod2>;
66     vector<ll> ans2 = NTT::multiply<mint2, mod2, root2
67 >(a, b);
68
69     int ans_size = a.size() + b.size() - 2;
70     ll M1_inv_M2 = mint<mod2>::inv(mod1).val;
71
72     vector<ll> final_result(ans_size + 1);
73     for (int i = 0; i <= ans_size; ++i) {
74         ll v1 = ans1[i];
75         ll v2 = ans2[i];
76         ll k = ((v2 - v1 + mod2) % mod2 * M1_inv_M2) %
77 mod2;
78         final_result[i] = v1 + k * mod1;
79     }
80     return final_result;
81 }
82 }

```

8.13 Euler's Totient

Returns the amount of numbers smaller than n that are coprime to n . Time: $\mathcal{O}(\sqrt{n})$

```

1 int phi(int n){
2     int ans = n;
3     for (int i = 2; i*i <= n; i++){
4         if (n%i == 0){
5             while(n%i == 0) n/=i;
6             ans -= ans/i;
7         }
8     }
9 }

```

```

8     }
9     if (n>1) ans -= ans/n;
10    return ans;
11 }

```

9 Geometry

9.1 Convex hull - Graham Scan

Time: $\mathcal{O}(n \log n)$

```

1 #define CLOCKWISE -1
2 #define COUNTERCLOCKWISE 1
3 #define INCLUDE_COLLINEAR 0 // pode mudar
4
5 struct Point {
6     ll x, y;
7     bool operator==(Point const& t) const {
8         return x == t.x && y == t.y;
9     }
10 };
11
12 struct Vec {
13     int x, y, z;
14 };
15
16 Vec cross(Vec v1, Vec v2){
17     int x = v1.y*v2.z - v1.z*v2.y;
18     int y = -v1.x*v2.z + v1.z*v2.x;
19     int z = v1.x*v2.y - v1.y*v2.x;
20     return {x,y,z};
21 }
22
23 ll dist2(Point p1, Point p2){
24     int dx = p1.x-p2.x;
25     int dy = p1.y-p2.y;
26     return dx*dx+dy*dy;
27 }
28
29 ll orientation(Point pivot, Point a, Point b){
30     Vec va = {a.x-pivot.x, a.y-pivot.y, 0};
31     Vec vb = {b.x-pivot.x, b.y-pivot.y, 0};
32     Vec v = cross(va,vb);
33     if (v.z < 0) return CLOCKWISE;
34     if (v.z > 0) return COUNTERCLOCKWISE;
35     return 0;
36 }
37
38 bool clock_wise(Point pivot, Point a, Point b) {
39     int o = orientation(pivot, a, b);
40     return o < 0 || (INCLUDE_COLLINEAR && o == 0);
41 }
42
43 bool collinear(Point a, Point b, Point c) { return
44 orientation(a, b, c) == 0; }
45
46 vector<Point> convex_hull(vector<Point> &points, bool
47 counterClockwise) {
48     int n = points.size();
49     Point pivot = *min_element(points.begin(), points.
50 end(), [](Point a, Point b) {
51     return ii(a.y, a.x) < ii(b.y, b.x);
52 });
53
54 sort(points.begin(), points.end(), [&](Point a,
55 Point b) {

```

```

52     int o = orientation(pivot, a, b);
53     if (o == 0) return dist2(pivot, a) < dist2(
54         pivot, b);
55     return o == CLOCKWISE;
56 });
57
58 if (INCLUDE_COLLINEAR) {
59     int i = n-1;
60     while (i >= 0 && collinear(pivot, points[i],
61         points.back())) i--;
62     reverse(points.begin()+i+1, points.end());
63 }
64
65 vector<Point> hull;
66 for (auto p : points) {
67     while (hull.size() > 1 && !clock_wise(hull[hull
68 .size()-2], hull.back(), p))
69         hull.pop_back();
70     hull.push_back(p);
71 }
72
73 if (!INCLUDE_COLLINEAR && hull.size() == 2 && hull
74 [0] == hull[1])
75     hull.pop_back();
76
77 if (counterClockwise && hull.size() > 1) {
78     vector<Point> reversed_hull = hull;
79     reverse(reversed_hull.begin() + 1,
80         reversed_hull.end());
81     return reversed_hull;
82 }
83
84 return hull;
85 }

```

9.2 Basic elements - geometry lib

- Basic elements for using the geometry lib, contains points, vector operations and distances between points, distance between point and segment, distance between segments, segment intersection check, orientation check (ccw).
- Always use long double for floating point. Only use floating point if indispensable.
- For $a == b$, use $|a - b| < \text{eps}$!!!!

Time: $\mathcal{O}(1)$

9.2.1 Polygon Area

- Heron's Formula for triangle area:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

, where a , b , and c are the triangle sides and $s = (a + b + c)/2$

TODO Shoelace

- Pick's Theorem for polygon area with integer coordinates:

$$A = a + b/2 - 1$$

, where a is the number of integer coordinates inside the polygon and b is the number of integer coordinates on the polygon boundary. b can be calculated for each edge as

$$b = \gcd(x_i + 1 - x_i, y_i + 1 - y_i) + 1$$

Polygon Area Time: $\mathcal{O}(n)$

9.2.2 Point in polygon

Sum of edge angles relative to the point must sum to 2π
Time: $\mathcal{O}(n \log n)$

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 typedef long double ld;
4 #define eps 1e-9
5 #define pi 3.141592653589
6 #define int long long int
7
8
9 struct pt {
10     int x, y;
11     int operator==(pt b) {
12         return x == b.x && y == b.y;
13     }
14     int operator<(pt b) {
15         if(x == b.x) return y < b.y;
16         return x < b.x;
17     }
18     pt operator-(pt b) {
19         return {x - b.x, y - b.y};
20     }
21     pt operator+(pt b) {
```

```
22         return {x+b.x, y + b.y};
23     }
24 };
25 int cross(pt u, pt v) {
26     return u.x * v.y - u.y * v.x;
27 }
28 int dot(pt u, pt v) {
29     return u.x * v.x + u.y * v.y;
30 }
31 ld norm(pt u) {
32     return sqrt(dot(u, u));
33 }
34 ld dist(pt u, pt v) {
35     return norm(u - v);
36 }
37 int ccw(pt u, pt v) { // cuidado com colineares!!!!
38     return (cross(u, v) > eps)?1:((fabs(cross(u, v)) <
39         eps)?0:-1);
40 }
41 int pointInSegment(pt a, pt u, pt v) { // checks if a
42     // lies in uv
43     if(ccw(v - u, a - u)) return 0;
44     vector<pt> pts = {a, u, v};
45     sort(pts.begin(), pts.end());
46     return pts[1] == a;
47 }
48 ld angle(pt u, pt v) { // angle between two vectors
49     ld c = cross(u, v);
50     ld d = dot(u, v);
51     return atan2l(c, d);
52 }
53 int intersect(pt sa, pt sb, pt ra, pt rb) { // not sure
54     // if it works when one of the segments is a point
55     pt s = sb - sa, r = rb - ra;
56     if(pointInSegment(sa, ra, rb) || pointInSegment(sb,
57         ra, rb) || pointInSegment(ra, sa, sb) ||
58         pointInSegment(rb, sa, sb)) return 1;
59     return !(ccw(s, ra - sa) == ccw(s, rb - sa) || ccw(
60         r, sa - ra) == ccw(r, sb - ra));
61 }
62
63 ld polygonArea(vector<pt>& p) { // not signed (for
64     // signed area remove the absolute value at the end)
```

```
65     ld area = 0;
66     int n = p.size() - 1; // p[n] = p[0]
67     for(int i = 0; i < n; i++) {
68         area += cross(p[i], p[i + 1]);
69     }
70     return fabs(area)/2;
71 }
72
73 int pointInPolygon(pt a, vector<pt>& p) { // returns 0
74     // for point in BOUNDARY, 1 for point in polygon and
75     // -1 for outside
76     ld total = 0;
77     int n = p.size() - 1;
78     for(int i = 0; i < n; i++) {
79         pt u = p[i] - a;
80         pt v = p[i + 1] - a;
81         if(fabs(dist(p[i], a) + dist(p[i + 1], a) -
82             dist(p[i], p[i + 1])) < eps) {
83             return 0;
84         }
85         total += angle(u, v);
86     }
87     return (fabs(fabs(total) - 2 * pi) < eps)?1:-1;
88 }
89
90 signed main() {
91     int n, m; scanf("%lld %lld", &n, &m);
92     vector<pt> p(n + 1);
93     for(int i = 0; i < n; i++) {
94         scanf("%lld %lld", &p[i].x, &p[i].y);
95     }
96     p[n] = p[0];
97
98     while(m--) {
99         pt a; scanf("%lld %lld", &a.x, &a.y);
100         int ans = pointInPolygon(a, p);
101         printf("%s\n", (ans > 0)?"INSIDE":(ans?"OUTSIDE"
102             ":"BOUNDARY));
103     }
104 }
```